

PPS - Scala Stream Fusion

Leonardo Naddei

24 settembre 2026

Indice

1	Introduzione	5
1.1	Obiettivi e criteri di valutazione	6
1.2	Struttura della relazione	6
2	Analisi dei requisiti	8
2.1	Requisiti di business	8
2.1.1	RB-1 Colmare il divario tra espressività e prestazioni	8
2.1.2	RB-2 Adozione a basso attrito	8
2.2	Requisiti funzionali utente	8
2.2.1	RFU-1 (Creazione dello Stream)	8
2.2.2	RFU-2 (Configurazione)	8
2.2.3	RFU-3 (Esecuzione parallela esplicita)	8
2.2.4	RFU-4 (Raccolta dei risultati)	9
2.2.5	RFU-5 (Guida contestuale tramite Type System)	9
2.3	Requisiti funzionali di sistema	9
2.3.1	RFS-1 (Riconoscimento e analisi della pipeline statica)	9
2.3.2	RFS-2 (Fusione delle operazioni sequenziali)	9
2.3.3	RFS-3 (Generazione di codice specializzato)	9
2.3.4	RFS-4 (Gestione del flusso e terminazione anticipata)	9
2.3.5	RFS-5 (Composizione di flussi annidati)	9
2.3.6	RFS-6 (Supporto all'esecuzione parallela)	9
2.4	Requisiti negativi ed esclusioni (Out of Scope)	10
2.4.1	RN-1 (Assenza di composizione dinamica runtime)	10
2.4.2	RN-2 (Nessuna ottimizzazione opaca sul codice utente)	10
2.4.3	RN-3 (Nessun supporto a backtracking o accesso casuale)	10
2.4.4	RN-4 (Nessun supporto a flussi annidati di stream)	10
2.4.5	RN-5 (Nessun fall-back silente a runtime)	10
2.4.6	RN-6 (Nessun inlining o ottimizzazione per funzioni non-lambda)	10
2.5	Requisiti non funzionali	10
2.5.1	RNF-1 (Prestazioni a runtime)	10
2.5.2	RNF-2 (Assenza di allocazioni intermedie)	10
2.5.3	RNF-3 (Preservazione della semantica)	11
3	Background e tecnologie abilitanti	12
3.1	Stream fusion	12
3.1.1	Push stream	12
3.1.2	Pull stream	13
3.2	Metaprogrammazione in Scala 3	13
3.2.1	Introduzione alla Multi-stage programming in Scala 3	13
3.3	Metaprogrammazione generativa e analitica	13
3.3.1	La reflection API e l'AST tipizzato	14

3.3.2	Tipi staged e tipi riflessivi	14
3.3.3	Beta reduction delle funzioni	15
3.4	Definizione di variabili	15
4	Design architetturale	16
4.1	Architettura generale	16
4.2	Componenti del sistema	16
4.2.1	Public API e DSL	16
4.2.2	Macro Entry Point	17
4.2.3	Parser	17
4.2.4	Stream IR	18
4.2.5	Enrichment	18
4.2.6	OP IR Generator	18
4.2.7	Lowering e Code Generator	18
4.3	Architettura del compilatore	19
4.4	Architettura runtime	19
4.4.1	Il modello scelto	19
4.4.2	Il runtime parallelo	20
5	Design di dettaglio	21
5.1	Design dell'API pubblica	21
5.1.1	Operazioni terminali	21
5.1.2	Creazione dello stream: <code>from</code>	23
5.1.3	Tipologie di stream	24
5.1.4	Operazione terminale <code>collect</code>	24
5.1.5	Configurazione compile-time e runtime	25
5.1.6	Import semplificato dell'API	25
5.2	Design della Stream IR	26
5.2.1	Struttura dell'albero	26
5.3	Rappresentazione per la Code Generation (OP Ir)	27
5.3.1	Valori	27
5.3.2	Operazioni	28
5.4	La collection strategy	29
6	Implementazione	30
6.1	Parsing	30
6.1.1	Parsing della stream	30
6.1.2	Parsing del collector	34
6.1.3	Parsing della configurazione statica	35
6.2	Enrichment	35
6.2.1	Materializzazione delle espressioni	36
6.2.2	Operatori sulla cardinalità	36
6.2.3	Propagazione delle condizioni di terminazione	36
6.2.4	Arricchimento di <code>flatMap</code>	37
6.2.5	Analisi della cardinalità	37
6.3	Traduzione in OP IR	38
6.3.1	Selezione della strategia terminale	38
6.3.2	Pianificazione della materializzazione in array	39
6.3.3	Generazione del corpo della pipeline	41
6.3.4	Generazione dei cicli sulle sorgenti	44
6.3.5	Specializzazione delle sorgenti	44
6.3.6	Generazione del piano parallelo	45

6.3.7	Risultato	45
6.4	Lowering	45
6.4.1	Lowering delle operazioni	45
6.4.2	Lowering dei valori	46
6.4.3	Conservazione delle informazioni di tipo	47
6.4.4	Applicazione e inlining delle funzioni	47
6.4.5	Lowering del parallelismo	47
6.4.6	Scheduling dei chunk	48
6.4.7	Risultato del lowering	48
6.5	Esempio di traduzione end-to-end	48
6.5.1	L'input alla libreria	49
6.5.2	Il risultato del parsing	50
6.5.3	L'output dell'enrichment	51
6.5.4	Generazione dell'OpIR	51
6.5.5	Lowering verso l'AST Scala	53
6.5.6	Il bytecode JVM generato	53
7	Validazione e valutazione	55
7.1	Strategia di testing	55
7.2	Analisi del codice generato	56
7.2.1	Fusione delle operazioni	56
7.2.2	Inlining delle lambda	56
7.2.3	Boxing e allocazioni	56
7.2.4	Analisi del bytecode	56
7.3	Valutazione delle prestazioni	56
7.3.1	Metodologia	56
7.3.2	Baseline	56
7.3.3	Benchmark sequenziali	56
7.3.4	Benchmark paralleli	56
7.3.5	Allocazioni	56
8	Retrospettiva	57

Abstract

Il presente lavoro descrive la realizzazione di una libreria per Scala 3 che consenta di definire pipeline di elaborazione secondo un paradigma dichiarativo, ottenendo al contempo prestazioni comparabili a quelle di codice imperativo specializzato manualmente.

Basandosi sul concetto di stream fusion, introdotto in *Stream Fusion: From Lists to Streams to Nothing at All*[1], e sfruttando le funzionalità di metaprogrammazione offerte dalle macro di Scala 3, è stata sviluppata una libreria capace di trasformare pipeline dichiarative in codice specializzato ed efficiente generato a compile-time. La libreria offre inoltre un'architettura estendibile per la definizione di nuovi operatori terminali, permettendo di integrare nuovi comportamenti nella fase di fusione senza rinunciare alle ottimizzazioni statiche.

Tale generazione statica permette di eliminare i principali overhead tipici delle astrazioni funzionali su JVM. L'analisi del bytecode prodotto conferma la completa fusione delle operazioni sequenziali in un unico ciclo imperativo e l'inlining diretto delle lambda nel callsite. Ciò azzerava l'uso di istruzioni `invokedynamic`, elimina il dispatch indiretto (evitando call site polimorfici o megamorfici) e rimuove le allocazioni di collezioni intermedie, iteratori e closure, azzerando al contempo il boxing dei tipi primitivi.

Per certi aspetti, il modello ottenuto richiama il paradigma di genericità dei template C++: il codice viene specializzato in fase di compilazione rispetto ai tipi e alla specifica composizione delle operazioni, quando tali informazioni sono disponibili staticamente. Le macro di Scala aggiungono tuttavia una componente esplicita di metaprogrammazione, permettendo non soltanto di istanziare codice generico, ma anche di analizzare, trasformare e ottimizzare la struttura della pipeline prima della sua emissione.

I benchmark condotti tramite JMH mostrano uno speedup significativo e un consumo di memoria drasticamente ridotto rispetto alle librerie standard. Il risultato è un framework che preserva l'espressività, la composizionalità e la type safety tipiche della programmazione funzionale.

Capitolo 1

Introduzione

Le API basate su pipeline costituiscono un modello di programmazione diffuso per l'elaborazione di collezioni e sequenze di dati. Operazioni come `map`, `filter` e `flatMap` permettono di descrivere una computazione in forma dichiarativa, separando la logica dell'elaborazione dai dettagli relativi all'attraversamento della struttura dati.

Questa forma di astrazione migliora la leggibilità del codice, ma può introdurre un costo rispetto a un'implementazione imperativa specializzata. Sulla JVM, una pipeline ad alto livello può richiedere la costruzione di iteratori o strutture intermedie, l'invocazione indiretta delle funzioni associate agli operatori e, in alcuni casi, operazioni di boxing e unboxing. Sebbene il compilatore JIT sia in grado di eliminare parte di questi costi, il risultato dipende dalle caratteristiche del programma e dal profilo osservato a runtime.

Il problema affrontato in questo progetto consiste quindi nel ridurre il divario tra due modalità apparentemente contrapposte di scrivere la stessa computazione: da un lato una pipeline dichiarativa. Dall'altro un ciclo imperativo scritto esplicitamente e specializzato per la specifica elaborazione da eseguire.

L'obiettivo di *FusedStream* è consentire di utilizzare il primo modello ottenendo, per quanto possibile, un codice eseguibile vicino al secondo.

La libreria sviluppata permette di descrivere pipeline di elaborazione attraverso un'API ispirata alle comuni API per stream e collezioni. Diversamente da una normale implementazione runtime, tuttavia, la pipeline non viene interpretata come una successione di oggetti e chiamate durante l'esecuzione. La sua struttura viene analizzata durante la compilazione mediante le funzionalità di metaprogrammazione di Scala 3 e trasformata in codice specializzato per la specifica composizione di sorgente, operatori e operazione terminale.

Il principio alla base della trasformazione è quello della *stream fusion*: le operazioni intermedie vengono fuse in un singolo flusso di esecuzione, evitando, quando possibile, la materializzazione dei risultati intermedi. L'analisi statica della pipeline consente inoltre di utilizzare informazioni relative ai tipi, alla sorgente, alla cardinalità del risultato e alla modalità di esecuzione per selezionare differenti strategie di generazione del codice.

Particolare attenzione è stata inoltre dedicata alla progettazione dell'API. Quando una proprietà della pipeline può essere espressa staticamente, la libreria la rappresenta attraverso il sistema di tipi invece di rimandarne la verifica alla fase di analisi o di esecuzione. La distinzione tra stream sequenziali e paralleli, le caratteristiche delle sorgenti e i requisiti delle operazioni terminali vengono quindi utilizzati per impedire già in fase di compilazione combinazioni non supportate.

Le operazioni terminali sono state progettate attraverso un'astrazione estensibile basata sui *collector*. Il sistema permette sia di utilizzare collector definiti dall'utente, sia di riconoscere alcune operazioni note alla libreria e sostituirle con strategie di generazione specializzate. Lo stesso modello viene esteso all'esecuzione parallela: quando sorgente, operatori e terminale pos-

siedono le proprietà necessarie, la computazione può essere partizionata in più unità di lavoro alle quali viene applicata la medesima pipeline fusa.

1.1 Obiettivi e criteri di valutazione

Il progetto può essere considerato efficace se riesce a preservare i principali vantaggi dell'interfaccia dichiarativa senza trasferirne il costo nel codice eseguito a runtime. In particolare, la valutazione del sistema considera i seguenti aspetti:

- **correttezza semantica:** il codice generato deve preservare il comportamento della pipeline descritta dall'utente;
- **fusione delle operazioni:** gli operatori intermedi devono essere incorporati, quando possibile, nello stesso attraversamento della sorgente senza produrre collezioni intermedie;
- **specializzazione del codice:** le informazioni disponibili a compile-time devono poter essere sfruttate per evitare strutture e dispatch non necessari e per selezionare strategie di esecuzione specifiche;
- **riduzione delle allocazioni:** l'elaborazione degli operatori intermedi non deve introdurre iteratori, closure o contenitori intermedi quando questi non sono semanticamente necessari;
- **prestazioni:** il throughput del codice generato deve essere confrontabile con quello di implementazioni imperative equivalenti e deve ridurre l'overhead rispetto alle corrispondenti pipeline offerte dalle librerie standard;
- **usabilità dell'API:** l'impiego della libreria deve richiedere una sintassi compatta e familiare e deve sfruttare il type system per impedire, per quanto possibile, configurazioni non supportate.

La validazione viene quindi condotta su più livelli. I test funzionali e di compilazione verificano rispettivamente la correttezza del comportamento e dei vincoli statici dell'API. L'ispezione del codice e del bytecode generato permette di verificare direttamente l'effettiva fusione delle operazioni e l'eliminazione degli overhead previsti. Infine, benchmark dedicati confrontano le prestazioni e le allocazioni della libreria con differenti implementazioni della stessa elaborazione.

1.2 Struttura della relazione

Il resto della relazione è organizzato come segue.

Il Capitolo 2 definisce i requisiti del sistema, distinguendo gli obiettivi rivolti all'utente, i requisiti interni del compilatore, i requisiti non funzionali e le funzionalità esplicitamente escluse dal progetto.

Il Capitolo 3 introduce i concetti necessari alla comprensione della soluzione, con particolare riferimento alla stream fusion e alle funzionalità di metaprogrammazione e reflection offerte da Scala 3.

Il Capitolo 4 presenta l'architettura complessiva di FusedStream e la suddivisione del processo di compilazione nelle sue principali componenti.

Il Capitolo 5 descrive le principali scelte di design, includendo l'API pubblica, la rappresentazione logica della pipeline, la rappresentazione intermedia utilizzata per la generazione del codice e il modello delle operazioni terminali.

Il Capitolo 6 approfondisce l'implementazione del compilatore, seguendo le trasformazioni che conducono dall'AST Scala iniziale alla Stream IR fino al lowering verso il codice Scala. Il capitolo

si conclude con un esempio di compilazione end-to-end che mostra le diverse rappresentazioni della stessa pipeline durante il processo di compilazione.

Il Capitolo 7 è dedicato alla validazione del sistema e alla valutazione delle prestazioni del codice generato.

Infine, il Capitolo 8 raccoglie una retrospettiva sul progetto, discutendone i limiti, le decisioni progettuali emerse durante lo sviluppo e i possibili sviluppi futuri.

Capitolo 2

Analisi dei requisiti

2.1 Requisiti di business

2.1.1 RB-1 Colmare il divario tra espressività e prestazioni

La programmazione funzionale favorisce la leggibilità, l'astrazione e la manutenibilità del codice attraverso l'impiego di pipeline dichiarative. Tuttavia, sulla JVM tali astrazioni comportano comunemente una penalizzazione prestazionale rispetto a cicli imperativi scritti a mano. L'obiettivo primario del progetto è annullare questo compromesso, consentendo di scrivere codice ad alto livello dichiarativo che compili direttamente in codice a basso livello massimamente efficiente.

2.1.2 RB-2 Adozione a basso attrito

L'integrazione della libreria all'interno di progetti esistenti non deve richiedere modifiche al toolchain standard del compilatore né l'apprendimento di API complesse o esotiche. La sintassi offerta deve risultare familiare a chi già utilizza le collezioni standard di Scala o le API di Java Stream, garantendo un'interfaccia compatta, uniforme e accessibile tramite un singolo import.

2.2 Requisiti funzionali utente

2.2.1 RFU-1 (Creazione dello Stream)

L'utente deve poter istanziare un flusso di elaborazione a partire da diverse sorgenti di dati (ad esempio collezioni Scala, array primitivi o iteratori) attraverso un unico punto di ingresso intuitivo (`FusedStream.from(...)`).

2.2.2 RFU-2 (Configurazione)

L'utente deve poter sovrascrivere il comportamento predefinito del sistema, sia per quanto riguarda i parametri di generazione statica del codice che per i parametri riguardanti l'esecuzione a runtime. In assenza di configurazione esplicita, il sistema deve utilizzare valori predefiniti senza richiedere ulteriori interventi da parte dell'utente.

2.2.3 RFU-3 (Esecuzione parallela esplicita)

Qualora la sorgente dei dati sia adatta, l'utente deve poter richiedere l'esecuzione concorrente della pipeline invocando esplicitamente il metodo `.parallel()`.

2.2.4 RFU-4 (Raccolta dei risultati)

L'utente deve poter raccogliere il risultato di una stream tramite `.collect(...)`, potendo utilizzare:

- Collector predefiniti della libreria (come `toList`, `toArray`, `summing`, `findFirst`);
- Collector personalizzati che rispettino l'interfaccia di accumulazione estendibile fornita dal framework.

2.2.5 RFU-5 (Guida contestuale tramite Type System)

L'API deve guidare attivamente l'utente durante la costruzione della pipeline sfruttando il type system. Tramite tipi e subtyping avanzato, l'IDE e il compilatore devono esporre ed accettare solo ed esclusivamente le operazioni valide per lo stato corrente dello stream (es. inibendo metodi non supportati in parallelo o impedendo l'uso di collector incompatibili con la policy di terminazione).

2.3 Requisiti funzionali di sistema

2.3.1 RFS-1 (Riconoscimento e analisi della pipeline statica)

Il sistema deve essere in grado di ispezionare l'albero sintattico di una pipeline dichiarativa e scomporla nella sequenza logica di sorgente, trasformazioni intermedie e operazione terminale a compile time.

2.3.2 RFS-2 (Fusione delle operazioni sequenziali)

Il sistema deve collassare le trasformazioni in un unico flusso di esecuzione continuo, eliminando la materializzazione di collezioni o iteratori tra uno stadio e il successivo.

2.3.3 RFS-3 (Generazione di codice specializzato)

Il sistema deve emettere codice imperativo target direttamente specializzato per i tipi di dato coinvolti e per l'operazione terminale richiesta, evitando chiamate polimorfiche o indirette.

2.3.4 RFS-4 (Gestione del flusso e terminazione anticipata)

Il sistema deve regolare il flusso di controllo del ciclo generato per supportare sia l'elaborazione esaustiva dell'intero flusso sia l'interruzione anticipata (*short-circuiting*) quando la logica dell'operazione terminale o di filtro lo richiede.

2.3.5 RFS-5 (Composizione di flussi annidati)

Il sistema deve consentire la fusione di flussi generati dinamicamente da elementi intermedi (`flatMap`), integrando il ciclo interno nel contesto del ciclo esterno senza creare buffer di accumulazione intermedi o computazioni superflue.

2.3.6 RFS-6 (Supporto all'esecuzione parallela)

Per le sorgenti e i terminali compatibili, il sistema deve emettere codice in grado di suddividere il carico di lavoro su più worker e aggregarne i risultati intermedi.

2.4 Requisiti negativi ed esclusioni (Out of Scope)

Al fine di garantire la terminazione del metaprogramma, la predicibilità prestazionale e la robustezza del compilatore, sono definiti i seguenti requisiti negativi:

2.4.1 RN-1 (Assenza di composizione dinamica runtime)

Il sistema non deve supportare la costruzione dinamica di pipeline determinate da rami condizionali ordinari a runtime (ad esempio `if (cond) s.map(...) else s.filter(...)`). L'intera catena di trasformazioni deve essere fissa e nota sintatticamente al call-site di compilazione.

2.4.2 RN-2 (Nessuna ottimizzazione opaca sul codice utente)

Il sistema non deve alterare né riscrivere la logica interna delle espressioni arbitrarie fornite dall'utente all'interno delle lambda, limitandosi all'inlining del corpo nel call-site generato.

2.4.3 RN-3 (Nessun supporto a backtracking o accesso casuale)

Gli stream sono modellati come flussi unidirezionali in avanti. Non è richiesto il supporto ad accessi casuali all'indietro o alla rilettura dello storico di elementi già consumati.

2.4.4 RN-4 (Nessun supporto a flussi annidati di stream)

Il sistema non supporta la generazione di stream i cui singoli elementi siano a loro volta istanze di stream tramite trasformazioni mapping (es. `map(x => FusedStream.of(x))`). La sola composizione nested consentita è quella finalizzata all'appiattimento esplicito tramite `flatMap`.

2.4.5 RN-5 (Nessun fall-back silente a runtime)

Se una composizione di stream viola i vincoli di tipo o non è compatibile con la fusione, il compilatore deve rigettare esplicitamente il programma tramite un errore di tipo statico o errore di compilazione della macro, senza ricorrere a conversioni lente o fallback trasparenti a runtime.

2.4.6 RN-6 (Nessun inlining o ottimizzazione per funzioni non-lambda)

Qualora a una trasformazione (es. `map`) venga passato una funzione preesistente anziché un'espressione lambda, il compilatore non applica la β -reduction né l'inlining statico.

2.5 Requisiti non funzionali

2.5.1 RNF-1 (Prestazioni a runtime)

Il codice imperativo generato deve esibire un throughput computazionale comparabile a cicli imperativi scritti manualmente in Java, riducendo l'overhead computazionale rispetto alle pipeline funzionali convenzionali ed azzerando il boxing introdotto dalla libreria per i tipi primitivi.

2.5.2 RNF-2 (Assenza di allocazioni intermedie)

L'elaborazione delle operazioni intermedie stateless non deve allocare oggetti effimeri sul runtime heap (nessun iteratore, nessuna closure associata a lambda, nessun contenitore intermedio).

2.5.3 RNF-3 (Preservazione della semantica)

A parità di sorgente, funzioni utente e configurazione, il codice generato deve produrre risultati semanticamente equivalenti a quelli definiti dalla pipeline dichiarativa, preservando l'ordine delle operazioni, la cardinalità del flusso e la semantica delle operazioni terminali.

Capitolo 3

Background e tecnologie abilitanti

Prima di poter passare alla descrizione dell'applicativo è necessaria, per poter comprendere il funzionamento, un introduzione al sistema di metaprogrammazione di Scala 3. Quanto segue è basato su [4, 5].

3.1 Stream fusion

Una pipeline di stream può essere rappresentata secondo due modelli di esecuzione fondamentali: un modello *push*, guidato dal produttore, e un modello *pull*, guidato dal consumatore. Questa distinzione viene discussa, nel contesto della stream fusion, da Kiselyov et al. in *Stream Fusion, to Completeness* [2].

3.1.1 Push stream

Nel modello *push* è la sorgente a guidare l'elaborazione. Ogni elemento prodotto viene immediatamente propagato attraverso gli operatori successivi della pipeline fino a raggiungere il terminale. Concettualmente, l'intera sequenza di operatori può quindi essere vista come una “mega-funzione” applicata a ciascun elemento della sorgente.

Per esempio, una pipeline della forma

```
1 stream
2 .filter(p)
3 .map(f)
4 .collect(collector)
```

può essere interpretata, come la composizione di una singola funzione:

$$\text{consume}(x) = \text{collector}(\text{map}(\text{filter}(x))).$$

Questa rappresentazione rende particolarmente semplice l'implementazione di operatori come **map** e **filter**: ciascun operatore scarta oppure inoltrare (eventualmente trasformandolo) l'elemento ricevuto senza modificare il meccanismo con cui la sorgente viene attraversata.

E' più complessa la gestione di operatori che modificano la cardinalità della stream. Operazioni come **skip** e **limit** richiedono infatti di mantenere uno stato relativo al numero di elementi osservati. Nel caso di **limit**, una volta raggiunto il numero di elementi, è necessario interrompere anticipatamente la produzione della sorgente.

Una difficoltà ancora maggiore emerge con operatori che combinano più stream come lo **zip**. Nel modello push ciascuna sorgente possiede infatti il proprio flusso di produzione: per associare l'elemento corrente di una stream con quello corrispondente di un'altra è necessario introdurre un meccanismo esplicito di sincronizzazione tra i due produttori.

3.1.2 Pull stream

Nel modello *pull*, al contrario, il controllo dell'esecuzione è affidato al consumatore. La stream viene rappresentata come un oggetto capace di produrre, su richiesta, il proprio elemento successivo. In questo caso non è la sorgente a inviare gli elementi verso gli operatori successivi, ma è il terminale a richiederli risalendo progressivamente la pipeline. Ogni operatore intermedio definisce quindi il proprio comportamento in termini dell'operatore precedente: una `map`, ad esempio, richiede un elemento alla stream sottostante e applica ad esso la funzione di trasformazione; un `filter` continua invece a richiedere elementi finché non ne incontra uno che soddisfa il predicato.

Questa organizzazione rende naturali operazioni come `limit`, poiché il consumatore può semplicemente cessare di richiedere nuovi elementi, e soprattutto operatori come `zip`: è sufficiente richiedere contemporaneamente un elemento a ciascuna delle due stream e combinarne i risultati. Il prezzo da pagare è una rappresentazione più esplicita dello stato dell'iterazione e, in assenza di successive ottimizzazioni, un maggior numero di passaggi intermedi necessari alla produzione di ciascun elemento.

3.2 Metaprogrammazione in Scala 3

3.2.1 Introduzione alla Multi-stage programming in Scala 3

Le due primitive fondamentali del sistema di metaprogrammazione staged di Scala 3 sono le *quote* e gli *splice*. Una quote `'{ expression }` costruisce una rappresentazione tipizzata del codice contenuto al suo interno. Se `expression` produce un valore di tipo `T`, il risultato della quote ha tipo `Expr[T]` e rappresenta codice che produrrà un valore di tipo `T` nello stage successivo.

Uno splice: `${expression}` opera invece nella direzione opposta e inserisce all'interno del codice quotato il frammento di programma rappresentato da un'espressione `Expr[T]`.

Un esempio di generazione staged è l'elevamento a potenza con esponente noto a compile time:

```
1 def powCode(x: Expr[Int], n: Int)(using Quotes): Expr[Int] =
2   if n == 0 then
3     Expr(1)
4   else
5     '{ $x * ${ powCode(x, n - 1) } }
```

La ricorsione di `powCode` viene eseguita durante l'espansione della macro, per un esponente pari a tre, il frammento generato è equivalente a `x * x * x * 1` e non contiene quindi cicli o ricorsioni.

3.3 Metaprogrammazione generativa e analitica

Una macro generativa costruisce un nuovo frammento di programma a partire da informazioni note durante la compilazione. E' è tuttavia necessario anche compiere l'operazione inversa: ricevere un frammento di programma, riconoscerne la struttura e decomporlo nei suoi componenti.

Scala 3 supporta questa forma di metaprogrammazione in maniera elementare attraverso il pattern matching sul codice quotato. Una generica espressione può essere analizzata con un pattern della forma

```
1 expr match {
2   case '{ someFunction($argument) } =>
3     ...
```

```
4 }
```

Nel progetto, tuttavia, è necessario accedere a dettagli più fini dell'albero generato dal compilatore, rendendo necessario il passaggio alla Reflection API.

3.3.1 La reflection API e l'AST tipizzato

Il tipo `Expr[T]` offre un'interfaccia ad alto livello sul codice. Quando è necessario analizzare direttamente il programma è possibile convertire una `Expr[T]` nel relativo nodo dell'AST del compilatore con `expr.asTerm`. La Reflection API espone la struttura tipizzata prodotta dal compilatore, tra i nodi più importanti utilizzati nel parser vi sono:

- `Apply`, per l'applicazione di una funzione;
- `TypeApply`, per un'applicazione contenente parametri di tipo;
- `Select`, per la selezione di un membro;
- `Ident`, per il riferimento a un identificatore;
- `Block`, per un blocco contenente dichiarazioni e un'espressione finale;
- `Inlined`, per rappresentare il risultato di un'espansione inline;
- `ValDef`, `Assign` e `Ref`, utilizzati durante la generazione di nuovo codice.

3.3.2 Tipi staged e tipi riflessivi

A livello di tipi Scala 3 mette a disposizione due rappresentazioni:

`Type[T]` rappresenta il tipo `T` nell'API staged ed è utilizzabile insieme a `Expr[T]`, quote e splice;

`TypeRepr` rappresenta invece un tipo all'interno della Reflection API e permette di ispezionarne direttamente la struttura.

Data una `TypeRepr`, il metodo `.asType` restituisce un valore `Type[?]` sul quale è possibile eseguire pattern matching:

```
1 item.asType match {  
2   case '[t] => ...  
3 }
```

Un esempio del suo utilizzo è l'estrazione del tipo per la creazione di un iteratore da un singolo elemento:

```
1 private def createSourceOf(item: Term): ParsedTree = {  
2   item.asType match {  
3     case '[t] =>  
4       val singleExpr = item.asExprOf[t]  
5       val iterableExpr: Expr[Iterable[t]] =  
6         '{ Iterable.single[t]({ singleExpr }) }  
7       ...  
8     }  
9   }
```

Questo esempio contiene in pochi passaggi l'intero collegamento tra reflection e staged programming:

1. viene determinato il tipo del termine dell'AST ricevuto;
2. il tipo viene estratto in `t` mediante il pattern `'[t]`;
3. il `Term` originale viene convertito in `Expr[t]`;
4. viene generato un nuovo frammento `Expr[Iterable[t]]`;

3.3.3 Beta reduction delle funzioni

Un altro elemento importante è la funzionalità di β -reduction. Data una funzione:

$$f = x \mapsto e, \tag{3.1}$$

L'applicazione

$$f(a) \tag{3.2}$$

Può essere riscritta sostituendo direttamente x con a all'interno di e . Scala 3 rende disponibile questa trasformazione attraverso: `Expr.betaReduce(...)`. Incorporando staticamente il corpo della lambda nell'IR generato, la macro elimina sia la rappresentazione della funzione come entità invocabile sia il relativo dispatch.

3.4 Definizione di variabili

Nell'AST di Scala, un riferimento a una variabile è associato al relativo `Symbol`. Il nome è solamente una delle proprietà del simbolo e non ne rappresenta l'identità. La Reflection API permette di creare nuove variabili attraverso: `Symbol.newVal(...)`, e di fare successivamente riferimento allo stesso simbolo attraverso `Ref(symbol)`. La definizione viene creata nell'AST tramite `ValDef(symbol, Some(initializer))`. La distinzione tra nome e simbolo garantisce inoltre l'*hygiene* della macro: variabili generate dalla macro non possono andare in conflitto con variabili omonime definite nel codice utente.

Capitolo 4

Design architetturale

4.1 Architettura generale

La libreria organizzata in due aree: l'*API pubblica*, e il *compilatore*.

L'API pubblica (descritta nella Sezione 5.1) definisce il modello di programmazione esposto dalla libreria: sorgenti, operatori intermedi e operazioni terminali. L'operazione terminale attiva, durante la compilazione, il processo di analisi e generazione statica del codice.

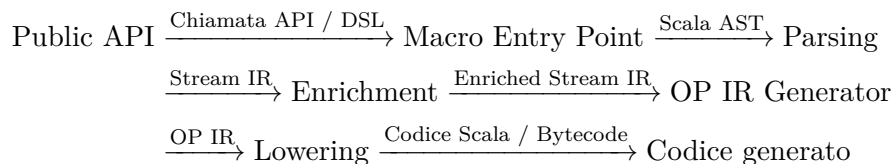
Il compilatore di FusedStream opera interamente a compile time: riceve dal compilatore Scala la rappresentazione sintattica tipizzata della pipeline, ne estrae una rappresentazione logica della pipeline, determina le informazioni necessarie alla sua esecuzione e produce un nuovo frammento di codice Scala. Quest'ultimo viene reinserito nel normale processo di compilazione e costituisce il codice effettivamente eseguito a runtime.

Si possono quindi distinguere due momenti chiari nel funzionamento del sistema:

- a *compile time*, la pipeline viene analizzata e trasformata in codice;
- a *runtime*, viene eseguito direttamente il codice generato, senza che sia necessario interpretare nuovamente la struttura della pipeline.

4.2 Componenti del sistema

L'architettura del sistema è organizzata come una pipeline che parte dal codice dell'utente e termina nella generazione del codice eseguibile. Come illustrato in Figura 4.1, ad alto livello il flusso principale può essere rappresentato come



4.2.1 Public API e DSL

La Public API (approfondita nella Sezione 5.1) costituisce il punto di ingresso del sistema per l'utente. Attraverso questa interfaccia vengono descritti: la sorgente, le operazioni da effettuare e l'operazione terminale.

La DSL non esegue direttamente le operazioni richieste, ma ha il solo scopo di descrivere le pipeline che può essere così osservata e trasformata dalla macro durante la compilazione.

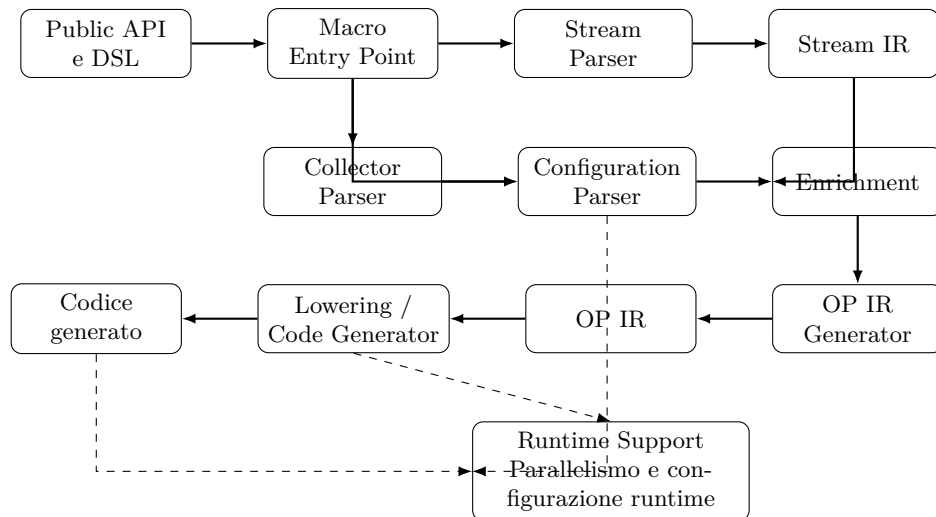


Figura 4.1: Architettura generale e principali componenti del sistema. Le frecce continue rappresentano il flusso di compilazione della pipeline, mentre le frecce tratteggiate indicano le dipendenze dal supporto runtime.

4.2.2 Macro Entry Point

Il Macro Entry Point rappresenta il confine tra il codice scritto dall'utente e il compilatore. Quando viene invocata un'operazione terminale, la macro riceve l'espressione che rappresenta la pipeline e avvia il processo di compilazione. Si occupa in particolare di orchestrare le diverse fasi, delegando ai componenti specializzati l'analisi e la trasformazione della stream:

- acquisisce l'AST della pipeline
- avvia il parsing della stream, dei collector e della configurazione
- invoca le fasi di enrichment e generazione delle rappresentazioni intermedie
- restituisce al compilatore Scala l'espressione finale generata

4.2.3 Parser

Il Parser traduce la rappresentazione sintattica della DSL in una rappresentazione logica: percorre l'AST e riconosce progressivamente operatori e sorgenti. Il risultato di questa fase è una struttura che rappresenta la stream dal punto di vista logico. La pipeline dell'utente:

```

1 FusedStream
2   .from(source)
3   .filter(p)
4   .map(f)

```

Diventa:

$\text{Map} \rightarrow \text{Filter} \rightarrow \text{Source}.$

Il parsing è effettuato anche sul terminale e sulla configurazione: il Collector Parser determina la strategia con cui gli elementi prodotti dalla stream devono essere consumati, il Configuration Parser analizza invece le informazioni che controllano la modalità di esecuzione della pipeline, come il grado di parallelismo o eventuali parametri del runtime.

Alcuni collector possono essere rappresentati direttamente da implementazioni fornite dall'utente, mentre altri costituiscono elementi speciali della DSL ai quali il compilatore associa una strategia specializzata.

4.2.4 Stream IR

La Stream IR (approfondita nella Sezione 5.2) è l'output del parser e costituisce la rappresentazione intermedia ad alto livello della pipeline. Ogni nodo conserva le informazioni necessarie a rappresentare l'operazione associata. La Stream IR costituisce quindi il punto di separazione tra il front-end della libreria, responsabile dell'interpretazione della DSL, e le fasi successive, responsabili della pianificazione e della generazione del codice.

4.2.5 Enrichment

La fase di Enrichment completa la Stream IR con informazioni non presenti nella descrizione iniziale della pipeline, ad esempio:

- proprietà della sorgente (es. accesso indicizzato);
- informazioni sulla cardinalità del flusso;
- caratteristiche degli operatori;
- possibilità di conoscere limiti o offset a compile-time;
- proprietà utilizzabili per scegliere strategie di generazione più efficienti.

4.2.6 OP IR Generator

L'OP IR Generator (approfondito nella Sezione 6.3) traduce la rappresentazione logica arricchita della stream nella rappresentazione intermedia orientata all'esecuzione (OP IR, Sezione 5.3). Se la Stream IR esprime quali operazioni compongono la pipeline, l'OP IR descrive come devono essere realizzate. La presenza di questa ulteriore rappresentazione permette di mantenere le decisioni relative al modello di esecuzione separate dai dettagli sintattici del linguaggio target.

4.2.7 Lowering e Code Generator

La fase di Lowering (approfondita nella Sezione 6.4) traduce l'OP IR nel codice Scala. Il Code Generator si occupa quindi di produrre espressioni corrispondenti a:

- dichiarazioni e assegnamenti;
- strutture condizionali;
- cicli;
- accessi ad array e collezioni;
- chiamate di funzione;

Il risultato finale viene restituito al compilatore Scala come AST e viene successivamente trattato come normale codice del programma.

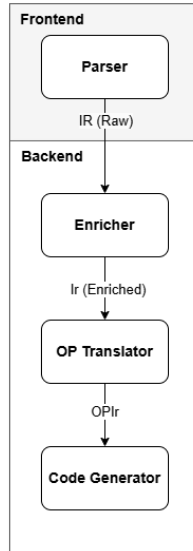


Figura 4.2: Architettura generale del compilatore di FusedStream.

4.3 Architettura del compilatore

L'architettura del compilatore è organizzata come una sequenza di trasformazioni che partono da un AST Scala e terminano con un, nuovo, AST Scala. Come mostrato in Figura 4.2, è possibile distinguere concettualmente il processo in un frontend e in un backend.

Il *frontend* ha il compito di analizzare l'espressione Scala della pipeline e di ricostruirne la struttura in una rappresentazione logica interna *Stream IR*. In questa fase la rappresentazione mantiene ancora una corrispondenza diretta con i concetti di alto livello del modello.

Il *backend* riceve la rappresentazione prodotta dal frontend e la trasforma progressivamente fino a ottenere il codice che implementa concretamente la pipeline. La prima fase consiste nell'*enrichment* della *Stream IR*. Durante questa trasformazione vengono associate all'albero informazioni necessarie alle fasi successive, tra cui variabili ausiliarie, condizioni di terminazione e proprietà statiche della pipeline. La struttura risultante è un *Enriched Stream IR*. La *Enriched Stream IR* conserva informazioni sufficienti per determinare una strategia di esecuzione. Essa viene quindi usata come base per produrre la *Operation IR* (OP IR). Questa è una sequenza di operazioni come dichiarazioni di variabili, assegnamenti, condizioni, cicli, accessi ad array. L'ultima fase è costituita dal *code generator*: consiste nella traduzione della OP IR verso costrutti Scala. A differenza di un compilatore tradizionale, il risultato è un nuovo AST Scala che viene restituito al compilatore Scala e reinserito nel normale processo di compilazione del programma.

Il flusso complessivo della trasformazione può quindi essere sintetizzato come:

Scala AST $\longrightarrow$ Stream IR_{Raw} $\longrightarrow$ Stream IR_{Enriched} $\longrightarrow$ OP IR $\longrightarrow$ Generated Scala AST.

4.4 Architettura runtime

4.4.1 Il modello scelto

Alla luce dei requisiti che non comprendono la necessità di effettuare zipping tra più stream e prediligono invece come obiettivo principe il massimo delle performance è stata naturale la scelta del modello *push* (Sezione 3.1.1).

Sintesi delle Operazioni supportate

In Tabella 4.1 viene riportato un quadro riassuntivo delle caratteristiche delle operazioni analizzate.

Tabella 4.1: Confronto delle operazioni sui flussi di dati.

Operazione	Firma	Tipo di Trasformazione	Cardinalità ($1 : M$)	Gestione Stato
Map	$A \rightarrow B$	$\mathcal{S}(A) \rightarrow \mathcal{S}(B)$	$1 : 1$	Stateless
Filter	$A \rightarrow \mathbb{B}$	$\mathcal{S}(A) \rightarrow \mathcal{S}(A)$	$1 : \{0, 1\}$	Stateless
Limit	$\mathbb{N}_0$	$\mathcal{S}(A) \rightarrow \mathcal{S}(A)$	$1 : \{0, 1\}$	Stateful
Skip	$\mathbb{N}_0$	$\mathcal{S}(A) \rightarrow \mathcal{S}(A)$	$1 : \{0, 1\}$	Stateful
FlatMap	$A \rightarrow \mathcal{S}(B)$	$\mathcal{S}(A) \rightarrow \mathcal{S}(B)$	$1 : N \ (N \geq 0)$	Stateless

4.4.2 Il runtime parallelo

L'esecuzione parallela mantiene lo stesso modello *push* adottato per l'esecuzione sequenziale. La parallelizzazione interviene sul modo in cui la sorgente viene attraversata: gli elementi vengono partizionati ed elaborati applicando la stessa pipeline fusa. Data la funzione P che rappresenta una generica pipeline:

$$P = o_n \circ o_{n-1} \circ \dots \circ o_1$$

E una sorgente S partizionabile in maniera efficiente in k elementi:

$$S = S_1 \cup S_2 \cup \dots \cup S_k,$$

l'esecuzione sequenziale è banalmente:

$$P(S),$$

Nel caso parallelo, invece dato $\oplus$ l'operatore che rappresenta la combinazione di risultati di una stream, possiamo rappresentare l'elaborazione come:

$$P(S_1) \oplus P(S_2), \dots, \oplus P(S_k).$$

A ciascuna partizione viene quindi applicato la stessa pipeline e i risultati intermedi vengono poi combinati. Alcuni combiner nativi nel caso parallelo non producono, strutture intermedie ma lavorano direttamente su memoria condivisa eliminando allocazioni intermedie

Operazioni possibili con stream parallele

La libreria rende possibile parallelizzare solamente con sorgenti e operatori per i quali il modello adottato consente di ottenere un'esecuzione prevedibilmente efficiente. Invece di permettere qualsiasi combinazione di sorgenti e operatori lasciando all'utente la responsabilità di valutarne l'effettiva convenienza, l'API impedisce la costruzione di pipeline parallele considerate inadatte.

La restrizione costituisce quindi una scelta progettuale e non una semplice mancanza di funzionalità. L'obiettivo è guidare l'utente verso configurazioni per le quali il parallelismo rappresenti effettivamente una strategia di esecuzione appropriata.

Capitolo 5

Design di dettaglio

5.1 Design dell'API pubblica

Particolare attenzione è stata posta nella progettazione dell'API pubblica della libreria. L'obiettivo è stato quello di fornire un'interfaccia che garantisca contemporaneamente:

- la possibilità di descrivere pipeline sufficientemente espressive
- l'estendibilità, in particolare per quanto riguarda le operazioni terminali
- il maggior numero possibile di garanzie a compile time
- la minimizzazione del codice necessario per verificare dinamicamente la correttezza delle combinazioni di operazioni richieste dall'utente.

Dal punto di vista concettuale, ogni stream può essere scomposto in tre componenti principali:

1. una sorgente, generalmente costituita da una collezione o da un array;
2. una sequenza di operazioni intermedie (`map`, `filter`, `flatMap`,...)
3. un'operazione terminale

La gerarchia di tipi esposta dall'API riflette direttamente questa struttura. Tramite di `trait sealed`, tipi parametrici, tipi opachi e vincoli sui parametri di tipo è stato possibile modellare le proprietà della pipeline. In questo modo il compilatore Scala può guidare l'utente durante la costruzione dello stream e impedire le combinazioni non supportate.

Allo stesso tempo, l'API mantiene un certo grado di estendibilità, in particolare consentendo all'utente di definire nuove operazioni terminali attraverso l'interfaccia del `Collector`.

5.1.1 Operazioni terminali

Le operazioni terminali sono rappresentate attraverso l'astrazione `Collector` (Figura 5.1):

I quattro parametri di tipo rappresentano rispettivamente:

- E: il tipo degli elementi ricevuti dallo stream;
- B: il tipo dello stato intermedio, o buffer, utilizzato durante l'accumulazione;
- R: il tipo del risultato finale prodotto dal collector;
- S: la politica di terminazione del collector.

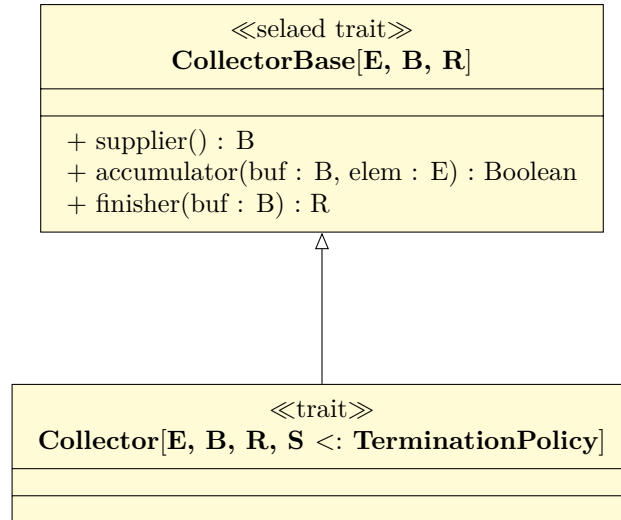


Figura 5.1: Gerarchia delle astrazioni `Collector` e `CollectorBase`.

Il metodo `supplier` crea lo stato iniziale dell'accumulator, `accumulator` aggiunge un nuovo elemento, il valore booleano restituito segnala che l'elaborazione può essere interrotta (si pensi a una `findFirst`). Infine, `finisher` trasforma lo stato intermedio nel risultato finale.

I collector possono essere esaustivi ovvero richiedere l'elaborazione dell'intera sorgente, oppure possono terminare anticipatamente senza consumare tutti i dati. Questa proprietà è rappresentata tramite la *TerminationPolicy* e i suoi due sottotipi (Figura 5.2):

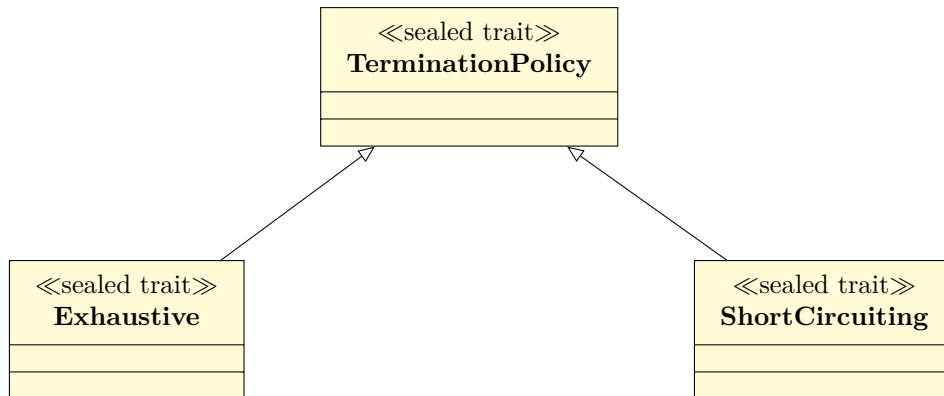


Figura 5.2: Gerarchia delle politiche di terminazione.

Ad esempio, i collector `toList` e `findFirst` hanno *TerminationPolicy* rispettivamente *Exhaustive* e *ShortCircuiting*. Nel caso del `toList` è necessario consumare tutti gli elementi dello stream prima di produrre la lista risultante. Nel secondo caso, invece, il primo elemento ricevuto è sufficiente a determinare il risultato e la pipeline è a quel punto interrotta.

Collector combinabili e parallelismo

Per poter eseguire una riduzione in parallelo è necessario ma non è sufficiente che il collector sia esaustivo. Gli i risultati intermedi prodotti indipendentemente dalle diverse partizioni dello stream devono anche poter essere combinati, questa proprietà viene rappresentata dall'interfaccia:

```

1 trait Combinable[B] {
2   def combine(left: B, right: B): B
3 }

```

Un collector utilizzabile come terminale di una pipeline deve essere quindi un sottotipo di *ParallelCollector* definito come:

```

1 trait ParallelCollector[E, B, R] extends Collector[E, B, R, Exhaustive] with
   Combinable[B]

```

Il tipo *ParallelCollector* esprime staticamente due le due proprietà richieste:

1. la necessità di elaborare completamente lo stream;
2. la possibilità di combinare i risultati intermedi ottenuti da più worker.

Il type system garantisce quindi la corretta compilazione della pipeline.

Collector specializzati

Alcune operazioni terminali particolarmente comuni dispongono di un trattamento specializzato da parte della libreria. Due esempi sono *toArray* e *summing*:

```

1 opaque type ToArrayCollector[T] <: ParallelCollector
2 opaque type SummingCollector[T <: Summable] <: ParallelCollector

```

Questi collector non vengono utilizzati come normali implementazioni runtime di *Collector*. La loro presenza nella pipeline viene riconosciuta e utilizzata per selezionare una strategia di code generation dedicata. Il tipo opaco permette di esporre le proprietà rilevanti. Ad esempio, entrambi risultano essere sottotipi di *ParallelCollector*, senza esporre all'utente i dettagli della strategia utilizzata internamente dal compilatore.

La libreria offre anche altri collector predefiniti come *toList* e *toSet* questi sono presenti solo per comodità e hanno lo stesso funzionamento di un collector che un qualsiasi utente potrebbe definire.

5.1.2 Creazione dello stream: *from*

La costruzione di una pipeline avviene a partire dal metodo *from*. Oltre a identificare la sorgente degli elementi, il tipo restituito da *from* determina quali modalità di esecuzione (parallela/sequenziale) possono essere richieste successivamente.

Non tutte le sorgenti possiedono infatti le caratteristiche necessarie per essere sfruttate parallelamente. L'API rappresenta questa proprietà attraverso il tipo restituito: una sorgente per la quale è disponibile l'esecuzione parallela viene rappresentata come un *ParallelizableSource[A]* (Figura 5.3).

Il metodo *parallel* non appartiene quindi genericamente a ogni *Stream*, ma solamente alle sorgenti per le quali la libreria dichiara esplicitamente il supporto alla parallelizzazione. Una pipeline costruita a partire da una sorgente non parallelizzabile non può essere convertita in una *ParallelStream*, l'operazione semplicemente non è disponibile nel tipo statico restituito all'utente.

5.1.3 Tipologie di stream

La modalità di esecuzione della pipeline è rappresentata attraverso una gerarchia di tipi **sealed**:

```
1 sealed trait Stream[A]
2 sealed trait SequentialStream[A] extends Stream[A]
3 sealed trait ParallelizableSource[A] extends SequentialStream[A]
4 sealed trait ParallelStream[A] extends Stream[A]
```

I sottotipi ridefiniscono il tipo restituito dalle operazioni. Una chiamata a **map** su una **ParallelStream[A]** produce quindi una **ParallelStream[B]**, impedendo che l'informazione relativa alla modalità di esecuzione venga persa lungo la pipeline.

La posizione di **skip** e **limit** esclusivamente all'interno di **SequentialStream** garantisce che queste operazioni non risultino disponibili su una **ParallelStream**.

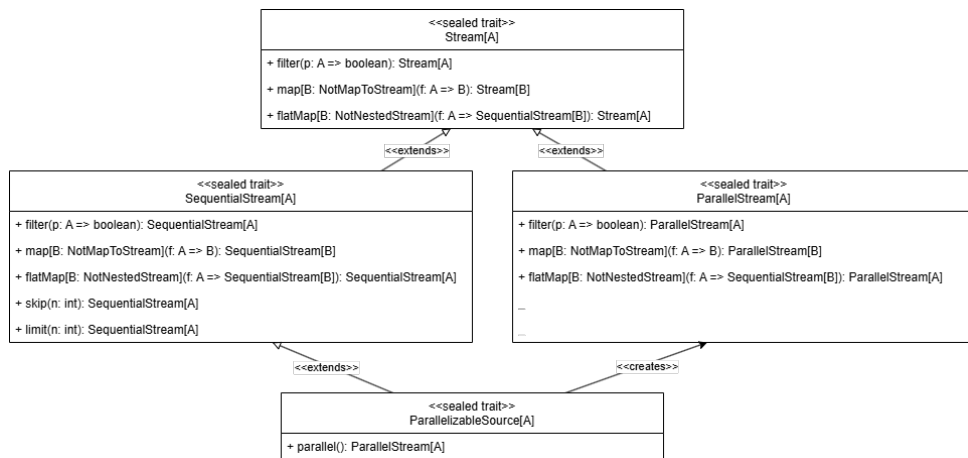


Figura 5.3: Diagramma UML delle classi per la gerarchia di Stream.

5.1.4 Operazione terminale collect

L'operazione **collect**, definita tramite extension methods, costituisce il punto nel quale la descrizione della pipeline viene trasformata nel codice che ne realizza l'esecuzione. La distinzione statica tra **SequentialStream** e **ParallelStream** permette di imporre requisiti differenti sul collector terminale.

Una pipeline sequenziale può utilizzare genericamente un **Collector**, mentre una pipeline parallela richiede un **ParallelCollector**, cioè un collector esaustivo e dotato di una funzione di combinazione dello stato intermedio. Le firme corrispondono quindi a:

```
1 // Sequential stream
2 extension [A](inline self: SequentialStream[A])
3   inline def collect[Buf, R, S <: TerminationPolicy](inline collector: Collector[A,
4     Buf, R, S])
5
6 // Parallel stream
7 extension [A](inline self: ParallelStream[A])
8   inline def collect[Buf, R](inline collector: ParallelCollector[A, Buf, R])
```

Questa distinzione permette di rifiutare espressioni non valide come l'utilizzo di un collector short-circuiting non combinabile come terminale di una pipeline parallela.

Ad esempio, un'operazione come:

l'utente debba conoscere o importare esplicitamente i diversi oggetti interni che compongono l'API.

5.2 Design della Stream IR

L'obiettivo della *Stream Intermediate Representation* (Stream IR) è rappresentare la struttura logica di una pipeline e, permettere alle successive fasi del compilatore di associare all'albero nuove informazioni.

Il design della Stream IR è una struttura ad albero estensibile che prende ispirazione dal paper *Trees That Grow* [3].

Il problema principale da gestire è che un Abstract Syntax Tree non rimane immutato durante l'intero processo di compilazione. Al contrario, le fasi del compilatore introducono informazioni da associare ai nodi dell'albero. Una possibile soluzione consiste nel definire una rappresentazione differente per ciascuna fase. Questo porterebbe però duplicazione di una parte considerevole della struttura dell'AST. Un'altra opzione potrebbe essere avere un nodo unico per ogni fase con campi opzionali o un nodo base con varie estensioni per reappresentare le proprie fasi, questi approcci hanno però un problema: rendono più complicata e soggetta a possibili errori la gestione di questi dati.

E' stata invece creata una struttura parametrizzata a livello di tipo P per determinare fase alla quale appartiene l'albero.

```
1 sealed trait Phase
2 object Phase {
3   sealed trait Raw extends Phase
4   sealed trait Enriched extends Phase
5 }
6 enum StreamTree[P <: Phase, A] {
7   ...
8 }
```

A Rappresenta il tipo degli elementi prodotti dal nodo.

P Identifica la fase del ciclo di vita (**Phase**) nella quale il nodo si trova.

L'utilizzo della fase come parametro di tipo, rende esplicito al type system quali rappresentazioni possono essere manipolate, ne derivano benefici come pattern matching esaustivo e garanzie a compile time.

Il passaggio tra il risultato del parser e dell'enricher quindi viene espresso direttamente attraverso i tipi.

```
StreamTree[Phase.Raw, A] StreamTree[Phase.Enriched, A].
```

5.2.1 Struttura dell'albero

La Stream IR modella una pipeline come un albero i cui nodi corrispondono alle operazioni del dominio degli stream. Le sorgenti costituiscono la foglia dell'albero.

Una rappresentazione semplificata della gerarchia è mostrata in Figura 5.5.

Gli operatori che non modificano la propria rappresentazione tra le due fasi sono parametrizzati direttamente tramite P , Ad esempio Map:

```
1 case Map[P <: Phase, A, B](upstream: StreamTree[P, A], ...) extends StreamTree[P, B]
```

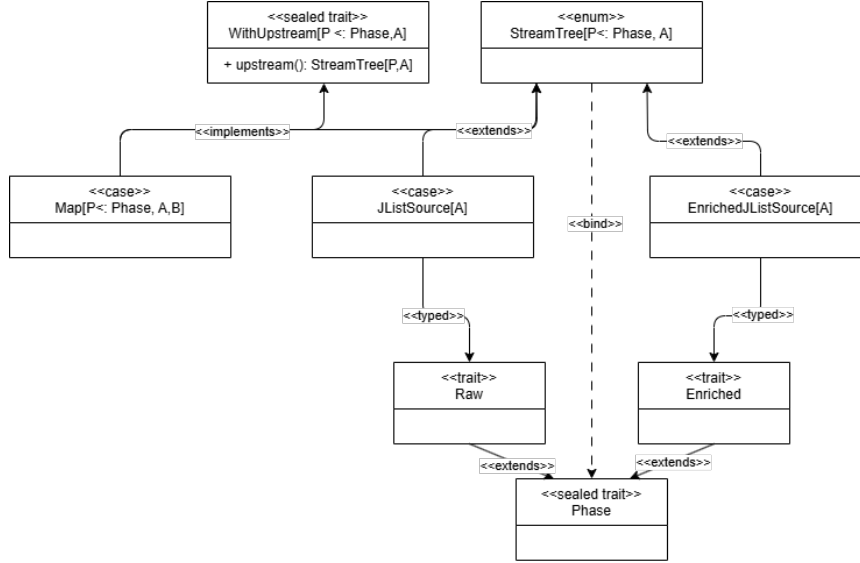


Figura 5.5: Struttura semplificata della Stream IR.

Si noti come la fase del nodo impone anche la fase del relativo *upstream* garantendo la correttezza dell'intera struttura dati

Gli operatori la cui struttura viene invece effettivamente estesa hanno varianti specifiche per fase. Un esempio dedicato al `flatMap` è riportato nella sezione a seguire.

5.3 Rappresentazione per la Code Generation (OP IR)

La Stream IR descritta nella sezione precedente mantiene una rappresentazione di alto livello della pipeline e dei suoi operatori. Tale struttura è particolarmente adatta alle analisi specifiche del dominio degli stream, ma non costituisce una rappresentazione conveniente per la generazione del codice.

Per questo si introduce una seconda rappresentazione intermedia, *Operation IR* (OP IR). La traduzione verso questa rappresentazione costituisce un processo di *lowering*: operatori di alto livello vengono eliminati e sostituiti da costrutti che descrivono esplicitamente il modello di esecuzione della pipeline. A differenza della Stream IR, la OP IR non modella quindi il dominio degli stream.

La rappresentazione è organizzata attorno a due categorie fondamentali: *Values*, che descrivono espressioni che producono un valore, e *Operations*, che descrivono azioni e strutture di controllo. Un programma nella OP IR è una sequenza di operazioni e un valore che ne costituisce il risultato finale.

Questo ulteriore linguaggio intermedio permette di suddividere la generazione del codice in due fasi con responsabilità distinte. La prima opera sulla rappresentazione logica della pipeline e, sulla base degli operatori presenti, della sorgente, dell'operazione terminale e delle proprietà ricavate durante l'analisi, come la cardinalità e l'allineamento degli indici, determina la strategia di esecuzione più adatta e la esprime attraverso la OP IR. La seconda fase è invece meccanica: opera su un insieme ristretto di primitive della OP IR e ne effettua il *lowering* nei corrispondenti costrutti Scala. Le decisioni relative a come implementare la pipeline sono quindi prese prima della generazione effettiva del codice.

5.3.1 Valori

I *Values* rappresentano espressioni che producono un risultato e sono utilizzati come operandi dalle operazioni della OP IR. La gerarchia è composta da innanzitutto valori elementari, come

costanti o riferimenti a Simboli e da valori composti che descrivono operazioni su altri valori come confronti, operazioni aritmetiche o applicazioni di funzioni, sono inoltre presenti valori specifici come accessi a array o iteratori.

Un **Value** descrive quindi un'espressione che può essere incorporata all'interno di una **Op**. Ad esempio, la condizione di un ciclo **While** è rappresentata come un **Value[Boolean]**, mentre il valore assegnato da una **AssignVal** è a sua volta rappresentato tramite un **Value**. Un esempio parziale della gerarchia è riportato in Figura 5.6.

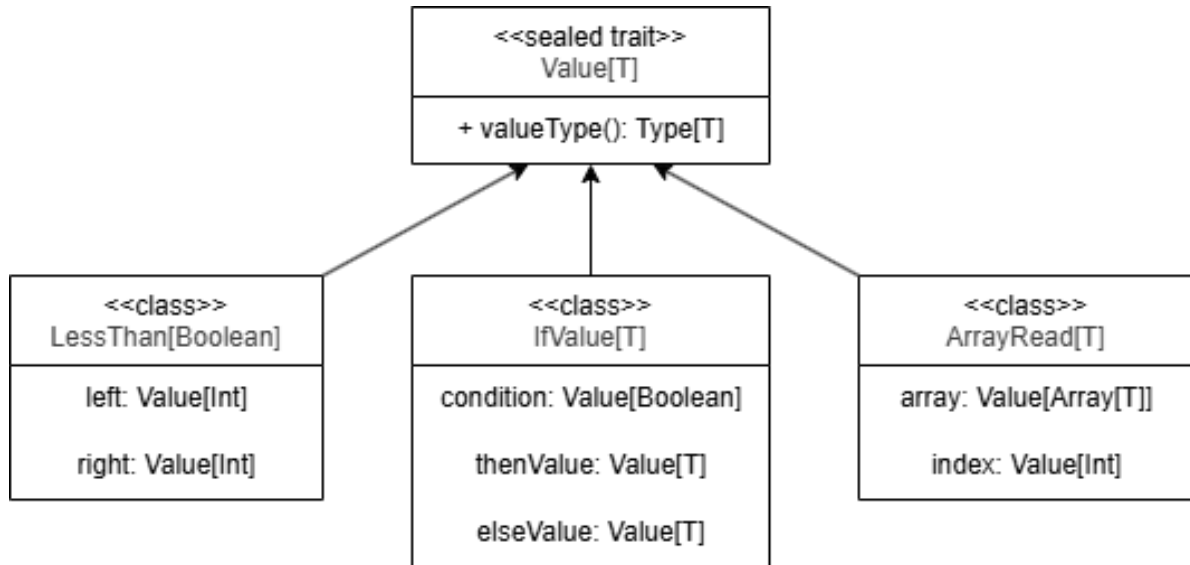


Figura 5.6: Gerarchia e rappresentazione dei valori nella OP IR.

Ogni valore possiede l'informazione relativa al tipo che rappresentano, permette quindi di esprimere staticamente i vincoli, a livello di tipi tra le diverse primitive della IR. Per esempio in **LessThan**, entrambi gli operandi devono essere valori interi, mentre il risultato è necessariamente un **Value[Boolean]**.

La sola informazione contenuta nel parametro generico non è però sufficiente per la successiva generazione del codice, ogni **Value[T]** espone quindi anche un **Type[T]**, attraverso il campo **valueType**.

Si realizza così una forma di *type capture*: oltre a conoscere staticamente che un nodo produce un valore di tipo **T**, la IR conserva una rappresentazione del tipo utilizzabile dalle Scala Quotes. Per tipi noti in maniera statica, per valori parametrici viene invece propagata attraverso un parametro contestuale **Type[T]**.

Come approfondito nella Sezione 6.4.3, questa informazione viene poi sfruttata durante il lowering per costruire espressioni Scala correttamente tipizzate.

5.3.2 Operazioni

Le *Operations* rappresentano invece le azioni che costituiscono il corpo del programma generato.

La OP IR comprende operazioni elementari per la dichiarazione e l'aggiornamento dello stato, come **Declare**, **AssignVal** o **Inc**. Il controllo del flusso è espresso esplicitamente attraverso **If** e **While**.

Sono inoltre presenti operazioni specializzate per le strategie di materializzazione utilizzate dalla libreria. **ArrayWrite**, ad esempio, rappresenta direttamente la scrittura di un elemento in un array, mentre **DynamicArrayBuilderAdd** rappresenta l'inserimento in un buffer la cui dimensione non è nota staticamente.

5.4 La collection strategy

La fase terminale di una stream è rappresentata internamente attraverso una `CollectionStrategy` (Figura 5.7). Questa astrazione descrive la strategia con cui gli elementi prodotti dalla pipeline devono essere consumati e aggregati per ottenere il risultato finale.

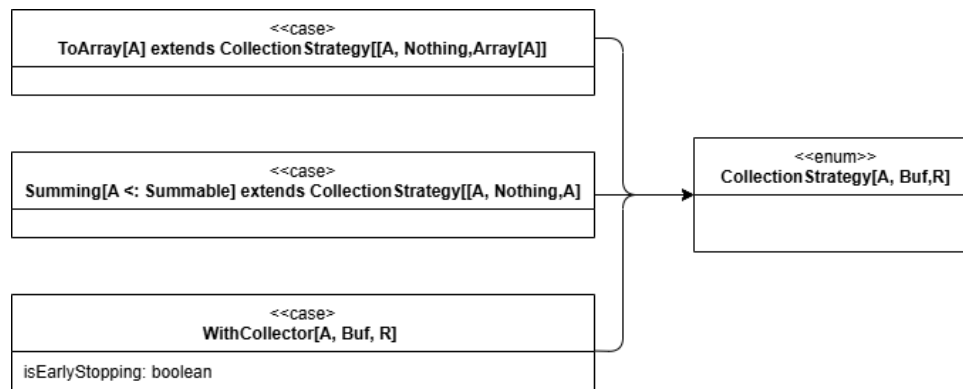


Figura 5.7: Diagramma UML delle classi per la gerarchia `CollectionStrategy`.

L'introduzione di questa rappresentazione permette di separare il collector espresso dall'utente nella DSL dalla modalità con cui il compilatore genera il codice relativo all'operazione terminale. In particolare, i collector possono essere suddivisi in due categorie.

La prima comprende i *collector concreti*, che forniscono direttamente l'implementazione delle operazioni necessarie alla raccolta. Tali operazioni vengono quindi invocate dal codice generato durante l'esecuzione della stream.

La seconda comprende invece i *collector opachi*. Questi non costituiscono l'implementazione effettivamente utilizzata a runtime, ma agiscono come placeholder tipati all'interno della DSL. Il compilatore li riconosce durante il parsing e li sostituisce con una `CollectionStrategy` specializzata, la cui implementazione è definita direttamente all'interno della libreria. Appartengono a questa categoria, ad esempio, i collector utilizzati per le operazioni `toArray` e `summing`.

Questa distinzione consente quindi di specializzare le operazioni terminali note alla libreria, evitando di rappresentarle necessariamente attraverso il protocollo generale di un collector.

Capitolo 6

Implementazione

6.1 Parsing

La fase di parsing ha l'obiettivo di trasformare l'albero sintattico tipizzato prodotto dal compilatore Scala in una `StreamIR` (Sezione 5.2) e di estrarre, a partire dal collector (Figura 5.4), la corrispondente `CollectionStrategy`. Il parsing della pipeline e quello dell'operazione terminale sono mantenuti separati e vengono successivamente combinati nella rappresentazione utilizzata dalle fasi successive.

6.1.1 Parsing della stream

Data una fused stream come:

```
1  val result = FusedStream
2    .from(numbers)
3    .filter(i => i % 2 == 0)
4    .skip(1)
5    .map(i => i * 10)
6    .collect(summing)
```

l'obiettivo della fase di parsing è ricondursi a una `StreamIr` del tipo:

$$\begin{aligned} \text{ArraySource[Int]} &\xrightarrow{\text{filter:}(Int \rightarrow \text{Boolean})} \text{Filter[Int]} \\ &\xrightarrow{\text{skip:}Int} \text{Skip[Int]} \\ &\xrightarrow{\text{map:}(Int \rightarrow Int)} \text{Map[Int, Int]} \end{aligned}$$

(si noti che il collector non è incluso in questa schematizzazione in quanto trattato separatamente nella Sezione 6.1.2)

Il parsing è effettuato analizzando l'AST tipizzato esposto dalla reflection API, in particolare il codice del Listato 6.1.1 produce l'albero mostrato nel Listato 6.1:

Listing 6.1: Rappresentazione ad albero dell'AST di Scala

```
1  Inlined(
2    None,
3    Nil,
4    Apply(
5      TypeApply(
6        Select(
7          Apply(
```

```

8      Select(
9          Apply(
10             Select(
11                 Apply(
12                     TypeApply(
13                         Select(Ident("FusedStream"), "from"),
14                         List(Inferred())
15                     ),
16                     List(Ident("numbers"))
17                 ),
18                 "filter"
19             ),
20             List(
21                 Block(
22                     List(
23                         DefDef(
24                             "$anonfun",
25                             List(TermParamClause(List(ValDef("i", Inferred(), None)))),
26                             Inferred(),
27                             Some(
28                                 Apply(
29                                     Select(
30                                         Apply(
31                                             Select(Ident("i"), "%"),
32                                             List(Literal(IntConstant(2)))
33                                         ),
34                                         "=="
35                                     ),
36                                     List(Literal(IntConstant(0)))
37                                 )
38                             )
39                         )
40                     ),
41                     Closure(Ident("$anonfun"), None)
42                 )
43             ),
44             "skip"
45         ),
46         List(Literal(IntConstant(1)))
47     ),
48     "map"
49 ),
50 List(Inferred())
51 ),
52 List(
53     Block(
54         List(
55             DefDef(
56                 "$anonfun",
57                 List(TermParamClause(List(ValDef("i", Inferred(), None)))),
58                 Inferred(),
59                 Some(
60                     Apply(
61                         Select(Ident("i"), "*"),
62                         List(Literal(IntConstant(10)))
63                     )
64                 )
65             )

```

```

66         )
67     ),
68     Closure(Ident("$anonfun"), None)
69 )
70 )
71 )
72 )

```

Rappresentazione dei nodi: `ParsedTree`

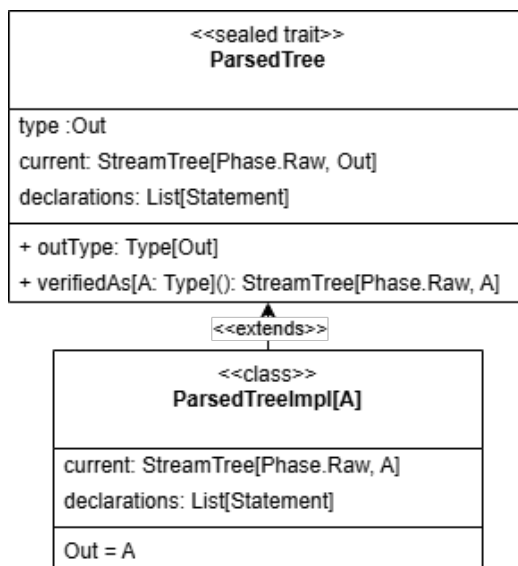


Figura 6.1: Struttura per parsed tree

Il trait `ParsedTree` (illustrato in Figura 6.1) rappresenta il risultato intermedio del parsing senza richiedere che il tipo di output sia noto al chiamante. Questo è necessario perché il tipo prodotto da una pipeline non è noto a priori, ma viene "scoperto" durante il parsing.

Per rappresentare questo tipo "nascosto", `ParsedTree` utilizza un abstract type member `Out`. Il tipo concreto viene fissato al momento della costruzione dell'implementazione, quando il tipo è noto. In questo modo il parser può esporre semplicemente un `ParsedTree`, mantenendo però internamente la correlazione tra il tipo dell'albero `current : StreamTree[Phase.Raw, Out]` e il relativo tipo di output.

Il tipo nascosto può essere utilizzato nelle successive fasi di parsing tramite un path-dependent type, ad esempio `type In = upstream.Out`. Il tipo precedentemente nascosto diventa così nuovamente utilizzabile per tipizzare operazioni successive, come una funzione `In => Boolean` per `filter` oppure `In => Out` per `map`.

Infine, `outType : Type[Out]` conserva l'evidenza reificata del tipo nascosto, necessaria per interagire con le API di `scala.quoted`, ad esempio per convertire un `Term` in una `Expr[In => Boolean]`.

Il riconoscimento delle espressioni

Il riconoscimento delle espressioni è effettuato tramite pattern matching

Listing 6.2: Pattern Matching sul Syntax Tree in Scala

```

1 private def parseTerm(expr: ir.quotes.reflect.Term): ParsedTree =
2   expr match

```

```

3 // In caso di espressioni inlined analizza l'espressione sottostante
4 case Inlined(_, _, inner) => parseTerm(inner)
5 case Apply(TypeApply(NamedMethod("from"), _), List(arr)) => ...
6 case Apply(TypeApply(NamedMethod("of"), _), List(arr)) => ...
7 case Apply(Select(upstream, "filter"), List(predicate)) => ...
8 case Apply(Select(upstream, "skip"), List(n)) => ...
9 case Apply(Select(upstream, "limit"), List(n)) => ...
10 case Apply(TypeApply(Select(upstream, "map"), List(outType)), List(mapper)) =>
11   ...
12 case Apply(TypeApply(Select(upstream, "flatMap"), List(outType)), List(mapper))
13   => ...
14 case Apply(Select(upstream, "parallel"), _) => parseTerm(upstream)
15 // In caso di un blocco accumula le dichiarazioni e continua con il parsing
16 case Block(blockStatements, expr) => parseTerm(expr) match {
17   case ParsedTreeImpl(tree, decls) => ParsedTreeImpl(tree, blockStatements ++
18     decls)
19 }

```

Esempi di parsing specifico: filter

Ogni caso poi tratta le informazioni ricevute ad esempio un filter è implementato come:

Listing 6.3: Parsing di un'operazione filter

```

1 case Apply(...) => appendFilter(parseTerm(upstream, predicate))
2
3 private def appendFilter(upstream: ParsedTree, predicateTerm: Term): ParsedTree =
4   {
5     type InOut = upstream.Out
6     given Type[InOut] = upstream.outType
7     val predicate = predicateTerm.asExprOf[InOut => Boolean]
8     val filter = Filter[Phase.Raw, InOut](upstream.current, predicate, upstream.
9       outType)
10    ParsedTreeImpl[InOut](filter, upstream.declarations)
11  }

```

Qui emerge un pattern ricorrente nel progetto: un tipo dipendente viene associato alla relativa evidenza reificata richiesta dalle API di `scala.quoted`. Il tipo di output del nodo precedente viene definito localmente tramite `type InOut = upstream.Out`. Viene quindi resa disponibile un'istanza `Type[InOut]` a partire da `upstream.outType`. In questo modo il predicato e l'albero `upstream.current` condividono staticamente lo stesso tipo `InOut`, senza ricorrere a cast.

Esempi di parsing specifico: flatMap

Un caso più complesso è dato da `flatMap`. A differenza `filter` la funzione passata a `flatMap` produce a sua volta una `Stream`. Tale stream deve quindi essere estratta dalla funzione e sottoposta ricorsivamente alla stessa procedura di parsing.

Si consideri, ad esempio:

```

1 FusedStream
2   .from(numbers)
3   .flatMap(x =>
4     FusedStream.from(elements(x)).filter(y => y > 0)
5   )

```

Il parametro `x` della funzione rappresenta l'elemento prodotto, a runtime, dall'upstream. Durante l'espansione della macro tale valore non è ancora disponibile; è tuttavia necessario per analizzare staticamente la stream interna. A questo scopo viene creato una variabile di binding di tipo `In`, ovvero il tipo di output dell'upstream:

```
1 type In = upstream.Out
2 given Type[In] = upstream.outType
3 val flatMapBinder = createConstantIn
4 val elemRef = Ref(flatMapBinder).asExprOf[In]
```

Il simbolo non contiene quindi il valore effettivo dell'elemento, ma costituisce un placeholder tipizzato che lo rappresenta all'interno della IR. La funzione ricevuta dal `flatMap` viene dapprima convertita nella corrispondente espressione tipizzata:

```
1 getType(outputType) match {
2   case '[out] =>
3     val function = functionTerm.asExprOf[In => Stream[out]]
4     ...
5 }
```

Quello che si ottiene è una funzione del tipo:

$$\text{In} \rightarrow \text{Stream}[\text{Out}].$$

A questo punto la funzione viene beta-ridotta (Sezione 3.3.3) contro il riferimento al binder:

```
1 val innerStreamExpr: Expr[Stream[out]] =
2   Expr.betaReduce {'{ $function($elemRef) }}
```

Ciò che si ottiene è quindi la stream interna che viene quindi sottoposta ricorsivamente a `parseTerm`. Si ottengono quindi due sottoalberi: l'upstream esterno e la stream interna prodotta dal `flatMap`. Il nodo della IR mantiene inoltre il simbolo del binder che collega i due: Durante la successiva fase di code generation il binder verrà associato al valore dell'elemento corrente prodotto dall'upstream. In questo modo la stream interna può essere analizzata e ottimizzata durante la compilazione pur dipendendo da un valore che sarà disponibile soltanto a runtime.

6.1.2 Parsing del collector

L'obiettivo di questa operazione è ricondursi a una collection strategy (approfondita nella Sezione 5.4) a partire dalla rappresentazione ricevuta. Il primo passo consiste nell'analizzarne il tipo associato all'espressione confrontandolo con i collector opachi della libreria, se è questo il caso il parser costruisce direttamente la strategia specializzata corrispondente. Nel caso generale, invece, il collector fornito dall'utente deve essere preservato, viene quindi estratta la *stop policy* per determinare staticamente se il collector supporti lo short-circuiting. Il risultato di questa analisi viene infine incorporato nella strategia generica `WithCollector`. La trasformazione può essere rappresentata come:

$$\text{ToArrayCollector}[A] \rightarrow \text{ToArray}[A]$$

$$\text{SummingCollector}[A] \rightarrow \text{Summing}[A]$$

$$\text{CollectorBase}[A, \text{Buf}, R] \rightarrow \text{WithCollector}[A, \text{Buf}, R]$$

Nota la tipizzazione su `TerminationPolicy` a questo punto viene persa, è infatti utilizzata solo per definire la grammatica della dsl e viene convertita nel valore booleano `isEarlyStopping` dentro `WithCollector`

6.1.3 Parsing della configurazione statica

La libreria effettua un'ulteriore operazione di parsing relativa alla configurazione di compilazione. A differenza del parsing della stream, basato sull'analisi del TASTy, in questo caso l'estrazione viene effettuata mediante quoted pattern matching. Poiché la configurazione deve essere interamente nota a compile time, i valori dei suoi campi devono poter essere estratti staticamente dall'espressione. Sono quindi accettate istanziazioni di `CompileConfig` i cui argomenti siano costanti note in fase di compilazione, come mostrato dal seguente `FromExpr`:

Listing 6.4: `Unapplier` con quoted pattern matching per configurazione statica

```
1 given FromExpr[CompileConfig] with {
2   def unapply(expr: Expr[CompileConfig])(using Quotes): Option[CompileConfig] = {
3     expr match {
4       // CompileConfig(unsafe, logging)
5       case '{ CompileConfig($u, $l) } =>
6         for {
7           unsafe <- u.value
8           logging <- l.value
9         } yield CompileConfig(unsafe, logging)
10
11      // new CompileConfig(unsafe, logging)
12      case '{ new CompileConfig($u, $l) } =>
13        for {
14          unsafe <- u.value
15          logging <- l.value
16        } yield CompileConfig(unsafe, logging)
17    }
18  }
19 }
```

6.2 Enrichment

La rappresentazione prodotta dal parser descrive la struttura logica della pipeline, ma non contiene ancora tutte le informazioni necessarie alla successiva generazione del codice. La fase di *enrichment* trasforma pertanto una `StreamTree[Phase.Raw, A]` nella corrispondente `StreamTree[Phase.Enriched, A]`, associando ai nodi informazioni determinate staticamente prima della costruzione della OP IR.

L'operazione può essere schematizzata come:

$$\text{Ast} \longrightarrow \text{AstExt},$$

dove la prima struttura contiene la `StreamTree` in fase `Raw`, mentre la seconda contiene l'albero arricchito e le informazioni ricavate dall'analisi:

```
1 case class AstExt[A, Buf, R](
2   enrichedStream: StreamTree[Phase.Enriched, A],
3   prefixStatements: List[Statement],
4   declarations: List[Declaration],
5   collectionStrategy: EnrichedCollectionStrategy[A, Buf, R],
6   hasAlignedIndexes: Boolean,
7   cardinality: Cardinality,
8   executionMode: ExecutionMode
9 )
```

La fase svolge quindi tre attività principali: materializza le espressioni che devono essere valutate una sola volta, propaga le condizioni che possono interrompere anticipatamente la produzione e determina proprietà globali della pipeline come la cardinalità del risultato e l'eventuale corrispondenza tra indice della sorgente e indice dell'elemento prodotto.

6.2.1 Materializzazione delle espressioni

Una prima responsabilità dell'enrichment consiste nell'estrarre le espressioni che devono essere valutate una sola volta prima dell'inizio dell'esecuzione. Un esempio è costituito dai parametri delle operazioni `skip` e `limit`, rappresentate internamente mediante il nodo `Slice`. I due estremi `from` e `until`, quando presenti, vengono materializzati in variabili immutabili, in questo modo un'espressione come:

```
1 stream.limit(computeLimit())
```

non provoca la rivalutazione di `computeLimit()` a ogni iterazione. Il valore viene calcolato una sola volta e le successive fasi operano esclusivamente sul riferimento alla variabile materializzata. Lo stesso principio viene applicato alle funzioni utilizzate da `map` e `filter`. Una lambda viene mantenuta nella propria forma originale, una funzione ottenuta tramite un'espressione invece materializzata in una variabile:

```
1 .map(computeMapper())
2
3 // Viene trasformata in
4 val mapper = computeMapper()
5 ....
6 .map(mapper)
```

6.2.2 Operatori sulla cardinalità

Le operazioni `skip` e `limit` vengono rappresentate nella Stream IR attraverso un unico nodo `Slice`. Durante l'enrichment a ciascun nodo viene associato un contatore rappresentante la posizione corrente rispetto al punto della pipeline nel quale è presente lo `Slice`. La sua materializzazione nella enriched IR permette di gestirne lo stato durante l'esecuzione.

Il caso di `limit` richiede inoltre un'informazione ulteriore. Una volta raggiunto il numero massimo di elementi, non è necessario continuare a interrogare la sorgente. Per questo motivo la condizione:

$$\text{sliceCounter} < \text{until}$$

viene aggiunta all'insieme delle condizioni di terminazione e propagata verso i nodi posti a monte.

6.2.3 Propagazione delle condizioni di terminazione

Le condizioni di terminazione sono rappresentate durante l'enrichment come una lista di `Expr[Boolean]`. Queste condizioni vengono propagate verso le sorgenti e vengono poi incorporate nelle condizioni dei cicli generati. Oltre che da `limit`, una condizione di questo tipo può essere introdotta da un collector *short-circuiting*. Nel caso di una `CollectionStrategy.WithCollector` marcata come *early-stopping*, viene creata una variabile mutabile inizializzata a `true`.

Il riferimento entra quindi nell'insieme dei predicati di terminazione. Quando il collector determina che il risultato è completo, la variabile viene portata a `false` e tutti i cicli interessati dalla condizione cessano quindi di produrre ulteriori elementi.

Dal punto di vista della successiva generazione, la condizione di una sorgente non è più solamente (nel caso di un'Array):

$$i < size,$$

ma la congiunzione tra la condizione propria della sorgente e tutti i predicati propagati:

$$C_{loop} = C_{source} \wedge p_1 \wedge p_2 \wedge \dots \wedge p_n.$$

6.2.4 Arricchimento di flatMap

La presenza di flatMap necessita di una gestione delle informazioni dedicata, il nodo contiene due distinti alberi: l'upstream della pipeline esterna e la stream interna prodotta dalla funzione.

Durante l'enrichment la stream interna viene elaborata ricorsivamente:

```
1 (innerEnrichedStream, innerDecls, innerAccumulatedPredicates) = enrich(fm.  
   innerStream, exitPredicates)
```

Le dichiarazioni prodotte internamente e i relativi predicati vengono conservati nel nodo EnrichedFlatMap:

```
1 EnrichedFlatMap(  
2   upstream      = enrichedUpstream,  
3   innerStream   = innerEnrichedStream,  
4   ...  
5   innerDeclarations = fm.innerDeclarations,  
6   innerMaterialized = innerDecls,  
7   predicates      = innerAccumulatedPredicates  
8 )
```

Lo stato appartenente alla stream interna deve essere creato nuovamente per ciascun elemento prodotto dalla stream esterna. Una variabile associata, ad esempio, a un limit interno alla funzione di flatMap non può essere dichiarata globalmente, ma deve essere inizializzata all'interno dell'iterazione che genera quella specifica stream.

Al contrario, una condizione introdotta a valle della flatMap deve poter arrestare sia l'iterazione interna sia la produzione della sorgente esterna. L'enrichment determina quindi le informazioni necessarie affinché la successiva fase di generazione possa posizionare dichiarazioni e condizioni nel corretto livello di annidamento.

6.2.5 Analisi della cardinalità

Una volta costruita la Stream IR arricchita viene effettuata una seconda analisi dell'albero per determinare due proprietà utilizzate nella scelta della strategia di generazione: la cardinalità del risultato e l'allineamento degli indici.

La cardinalità è descritta mediante:

```
1 enum Cardinality {  
2   case Exact(size: Expr[Int])  
3   case UpperBound(size: Expr[Int])  
4   case Unknown  
5 }
```

Una sorgente indicizzabile, come un array o una java.util.List, possiede una cardinalità esatta, una map non modifica tale proprietà, un filter, invece, può eliminare elementi ma non può aumentarne il numero degradando una cardinalità esatta ad un upperbound.

Le sorgenti genericamente iterabili non espongono invece una dimensione utilizzabile senza effettuarne l'attraversamento e sono quindi rappresentate mediante `Cardinality.Unknown`. Anche una `flatMap`, potendo produrre un numero arbitrario di elementi per ciascun elemento in ingresso, rende sconosciuta la cardinalità complessiva.

L'informazione sulla cardinalità viene affiancata dal flag `hasAlignedIndexes`. Esso indica se l'indice dell'elemento nella sorgente può essere utilizzato direttamente come indice del risultato. La proprietà vale inizialmente per array e liste indicizzabili e viene preservata da `map` e `limit`. Viene invece invalidata da operazioni come `filter`, `skip` e `flatMap`, che rompono la corrispondenza tra posizione letta e posizione prodotta.

Le due informazioni permettono alla fase successiva di selezionare strategie di esecuzione differenti. Una pipeline con cardinalità esatta e indici allineati può, scrivere direttamente nella posizione finale di un array preallocato. Quando è noto soltanto un limite superiore è invece necessario mantenere un indice di scrittura separato ed eventualmente troncato l'array al termine dell'elaborazione. Se la cardinalità è sconosciuta, è necessario ricorrere a una struttura di accumulazione dinamica.

6.3 Traduzione in OP IR

Una volta completata la fase di enrichment, la Stream IR contiene le informazioni necessarie per determinare come la pipeline debba essere concretamente eseguita. La fase successiva ha quindi il compito di tradurre la rappresentazione logica della stream nella *Operation IR* (OP IR), introdotta nella Sezione 5.3.

È in questa fase che vengono prese le principali decisioni relative al piano di esecuzione. Il codice generato può assumere forme differenti in funzione di proprietà note staticamente, tra cui:

- l'operazione terminale richiesta;
- la modalità di esecuzione sequenziale o parallela;
- la cardinalità della pipeline;
- la possibilità di mantenere allineati gli indici della sorgente e del risultato;
- le caratteristiche della sorgente.

La traduzione in OP IR costituisce quindi una fase di pianificazione dell'esecuzione: vengono selezionate tra più strategie semanticamente equivalenti quelle che consentono di sfruttare al meglio le informazioni disponibili a compile time. La successiva fase di lowering potrà invece limitarsi alla traduzione delle operazioni emesse.

6.3.1 Selezione della strategia terminale

Il primo livello decisionale riguarda l'operazione terminale. La `CollectionStrategy` ottenuta durante il parsing permette di distinguere tra operazioni per le quali la libreria possiede una strategia specializzata e collector generici. I casi gestiti sono:

- `ToArray`, per il quale il compilatore genera direttamente il meccanismo di materializzazione del risultato;
- `Summing`, per il quale viene generato un accumulatore numerico specializzato sul tipo degli elementi;
- `WithCollector`, che utilizza il protocollo generale `supplier/accumulator/finisher`.

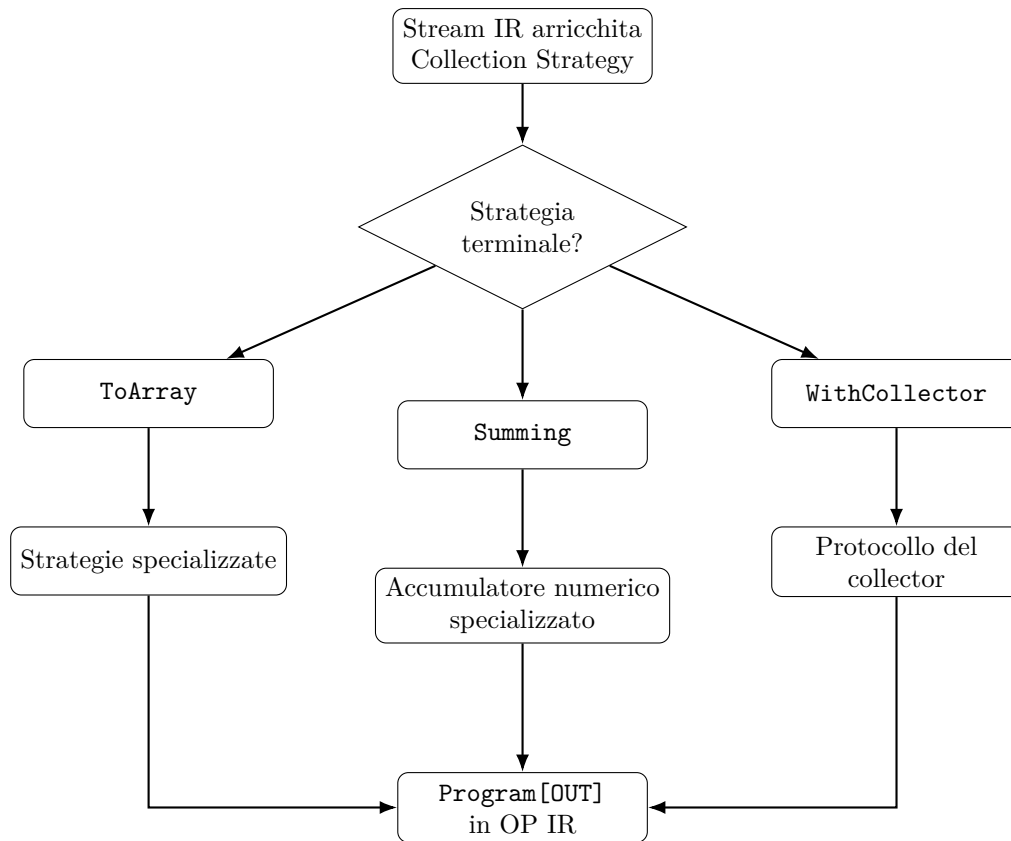


Figura 6.2: Selezione ad alto livello della strategia terminale durante la generazione della OP IR.

Il flusso decisionale ad alto livello è mostrato in Figura 6.2.

Ad esempio, una somma di valori `Int` viene rappresentata attraverso una variabile numerica inizializzata a zero e aggiornata direttamente per ciascun elemento prodotto:

```

1 Declare(sum, ConstantVal(0))
2 ...
3 AssignVal(sum, Add(SymbolRef(sum), elem))

```

Non è quindi necessario costruire un collector né invocarne l'accumulator per ogni elemento. Nel caso di un collector generico vengono invece generate esplicitamente le tre operazioni che costituiscono il relativo protocollo:

```

1 Declare(buffer, CollectorSupplier(collector))
2 ...
3 CollectorAccumulate(collector, buffer, elem)
4 ...
5 CollectorFinish(collector, buffer)

```

6.3.2 Pianificazione della materializzazione in array

Il caso di `toArray` mostra in maniera particolarmente evidente come le informazioni ottenute durante l'enrichment influenzino il piano di esecuzione prodotto.

Una prima discriminante è costituita dalla modalità di esecuzione, sequenziale oppure parallela. All'interno di ciascuna modalità vengono poi utilizzate la cardinalità della pipeline e l'informazione relativa all'allineamento degli indici.

La Figura 6.3 sintetizza le principali strategie.

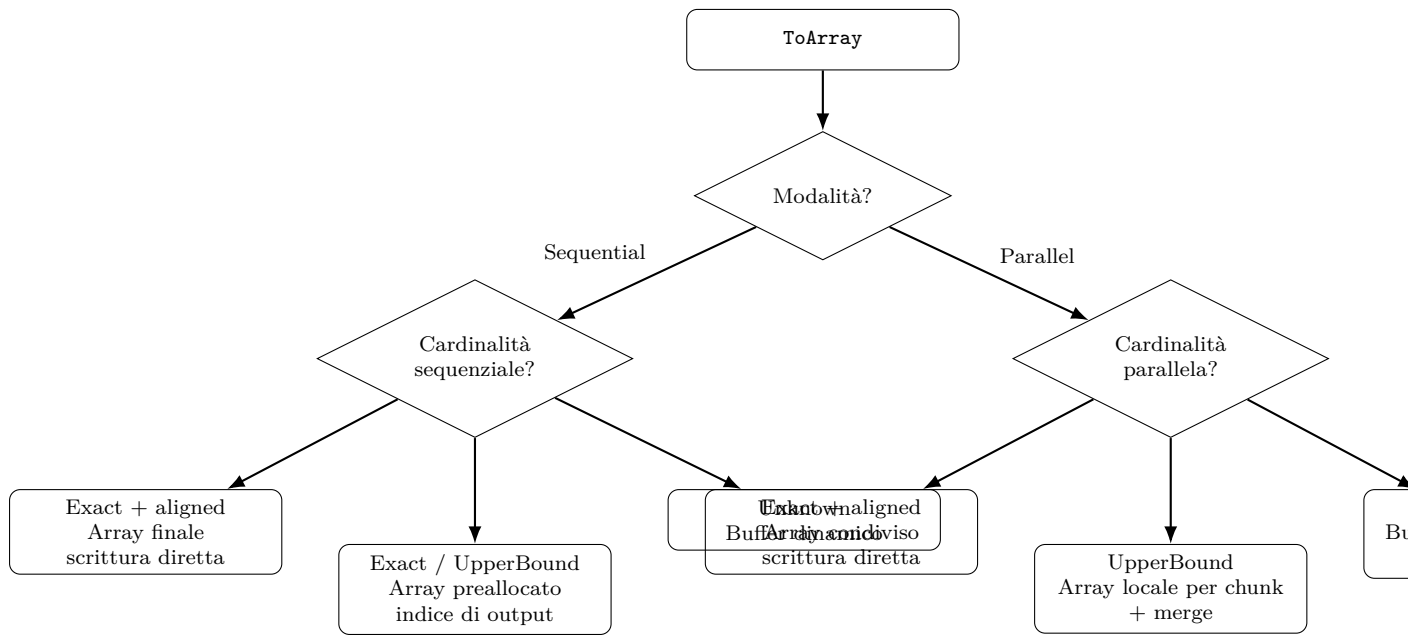


Figura 6.3: Selezione del piano di materializzazione per il terminale `toArray`.

Cardinalità esatta e indici allineati

Il caso più favorevole si presenta quando la cardinalità del risultato è nota esattamente e l'indice della sorgente coincide con quello dell'elemento prodotto.

Una pipeline costituita, ad esempio, esclusivamente da operazioni `map` su un array mantiene entrambe le proprietà. È quindi possibile allocare immediatamente il risultato della dimensione corretta:

```

1 Declare(
2   result,           // Dichiarare la costante result
3   ArrayDefine(size) // Inizializza con un array di dimensione nota a runtime
4 )

```

e utilizzare direttamente l'indice della sorgente durante l'emissione:

```

1 ArrayWrite(
2   SymbolRef(result), // Scrivi in result
3   emitted.sourceIndex.get, // Allo stesso indice dell'elemento nella sorgente
4   emitted.elem         // Scrivi l'elemento
5 )

```

Questa informazione risulta particolarmente utile anche nel caso parallelo. Poiché ciascun elemento possiede una posizione finale determinabile indipendentemente, i worker possono scrivere direttamente in regioni distinte dello stesso array finale. In questo caso il piano parallelo utilizza quindi la strategia `ParallelCombine.ArrayDirect()`, evitando completamente la costruzione e la successiva concatenazione di risultati locali.

Cardinalità limitata superiormente

La presenza di un'operazione come `filter` rompe la corrispondenza tra indice letto e indice prodotto. La dimensione della sorgente rimane tuttavia un limite superiore per quella del risultato.

È quindi ancora possibile preallocare un array, ma diventa necessario introdurre un contatore separato per la posizione di scrittura:

```
1 Declare(vec, ArrayDefine(size)) // Dichiaro un vettore con size nota
2 Declare(i, ConstantVal(0))      // Crea un contatore di elementi inseriti
3 ...
4 ArrayWrite(
5     SymbolRef(vec),
6     SymbolRef(i),
7     elem
8 )
9 Inc(i)
10 ...
11 // Al termine dell'elaborazione ritorna il vettore o una sua parte
12 // nel caso il numero di elementi emessi sia minore della size
13 IfValue(
14     Equal(SymbolRef(i), ArrayLength(SymbolRef(vec))),
15     SymbolRef(vec),
16     ArrayTake(SymbolRef(vec), SymbolRef(i))
17 )
```

La presenza di un limite superiore permette quindi di evitare il costo di un buffer dinamico, eventualmente sovraallocando, pur senza conoscere esattamente la dimensione finale.

Nel caso parallelo, la stessa informazione viene sfruttata localmente per ciascun chunk. Ogni partizione dispone di un array dimensionato secondo la propria porzione della sorgente e di un contatore che indica quanti elementi siano stati effettivamente prodotti. I risultati locali vengono poi concatenati utilizzando solamente la parte valida di ciascun array.

Cardinalità sconosciuta

Quando non è disponibile neppure un limite superiore affidabile, ad esempio in presenza di una sorgente genericamente iterabile o di un `flatMap`, non è possibile determinare preventivamente una dimensione appropriata per l'array. Viene quindi utilizzato un buffer dinamico:

```
1 Declare(
2     builder,
3     DynamicArrayBuilderNew()
4 )
```

e ciascun elemento viene aggiunto mediante:

```
1 DynamicArrayBuilderAdd(
2     SymbolRef(builder),
3     elem
4 )
```

Il risultato viene materializzato come array soltanto al termine dell'elaborazione. Anche nel caso parallelo il principio resta lo stesso: ogni chunk produce un proprio risultato locale, che viene successivamente concatenato agli altri risultati secondo l'ordine delle partizioni.

6.3.3 Generazione del corpo della pipeline

Una volta determinata la strategia terminale, deve essere generato il corpo che produce gli elementi da fornire all'accumulatore. La soluzione adottata utilizza un modello basato su una funzione di emissione. Un valore `Emitted[A]` contiene l'elemento prodotto e, quando disponibile, l'indice della sorgente:

```

1 private type Emit[A] = Emitted[A] => Op
2
3 private final case class Emitted[A](
4   elem: Value[A],
5   sourceIndex: Option[Value[Int]]
6 )

```

La funzione `Emit` rappresenta quindi una continuation del flusso: descrive cosa deve avvenire quando il nodo corrente produce un nuovo elemento.

Gli operatori intermedi non generano un ciclo proprio, ma costruiscono una nuova funzione `emit` che incorpora il proprio comportamento e delegano nuovamente la generazione all'upstream. Il ciclo viene creato solamente quando la ricorsione raggiunge la sorgente.

Traduzione di `filter`

Nel caso di un filtro, la continuation ricevuta viene racchiusa all'interno di una condizione. Dato E il generico comportamento del downstream, il filtro costruisce:

$$E'(x) = \text{if } p(x) \text{ then } E(x).$$

L'implementazione costruisce quindi:

```

1 Op.If(
2   ApplyFun(filter.predicate, emitted.elem),
3   emit(emitted)
4 )

```

e continua la generazione sull'upstream.

Traduzione di `map`

Una `map` modifica invece il valore che deve essere passato alla continuation.

La funzione viene applicata all'elemento ricevuto e il risultato viene materializzato in una variabile della OP IR:

```

1 val mappedSymbol = createConstant[OUT] ("mapped")
2
3 CodeBlock(
4   List(
5     Declare(mappedSymbol, ApplyFun(map.function, emitted.elem)),
6     emit(Emitted(SymbolRef(mappedSymbol), emitted.sourceIndex))
7   )
8 )

```

La materializzazione è importante quando il valore trasformato viene utilizzato più volte dai nodi downstream, ad esempio sia come argomento di un `filter` sia successivamente come valore. L'eventuale indice viene invece propagato invariato.

Traduzione di `slice`

Il nodo `Slice`, utilizzato internamente per rappresentare `skip` e `limit`, introduce uno stato associato alla posizione corrente.

Nel caso in cui sia presente un limite inferiore `from`, la continuation viene eseguita solamente quando il contatore ha raggiunto tale valore:

```

1 Op.If(
2   GreaterThanOrEqual(counter, from),
3   emit(emitted)
4 )

```

A ogni elemento ricevuto il contatore viene successivamente incrementato. Il limite superiore `until`, quando presente, è invece già stato propagato durante l'enrichment in una condizione di terminazione.

Traduzione di `flatMap`

La traduzione di `flatMap` è la più complessa: il nodo contiene una seconda Stream IR che rappresenta la pipeline interna. Durante il parsing è stato creato il binder che rappresenta l'elemento corrente prodotto dalla stream esterna; qui il simbolo viene associato al valore effettivo:

```

1 Declare(
2   flatMap.elemSymbol,
3   emitted.elem
4 )

```

La stream interna viene quindi tradotta ricorsivamente mediante la stessa `buildBody`, utilizzando come continuation quella ricevuta dal `flatMap`:

```

1 val innerBody =
2   buildBody(flatMap.innerTree, emit, flatMap.predicates, None)

```

Il corpo ottenuto viene inserito nella continuation utilizzata dalla stream esterna, preceduto dalle dichiarazioni e dall'associazione del binder con l'elemento della continuation:

```

1 CodeBlock(
2   List(
3     Declare(flatMap.elemSymbol, emitted.elem)
4   )
5   ++ innerDeclarations
6   ++ innerMaterialized
7   ++ List(innerBody)
8 )

```

Infine viene generato il corpo dell'upstream esterno:

```

1 buildBody(
2   flatMap.upstream,
3   upstreamEmit,
4   exitPredicates,
5   range
6 )

```

Il risultato è quindi un annidamento diretto dei due cicli. Una pipeline come:

```

1 source.flatMap(x =>
2   from(inner(x))
3   .filter(p)
4 )

```

viene trasformata in:

```

1 while (outerCondition) {
2   val x = nextOuter()
3
4   while (innerCondition) {
5     val y = nextInner(x)
6
7     if (p(y)) {
8       emit(y)
9     }
10  }
11 }

```

6.3.4 Generazione dei cicli sulle sorgenti

La ricorsione di `buildBody` termina quando viene raggiunta una sorgente. Per una sorgente come un array, il codice assume la forma:

```

1 Declare(i, from)
2
3 While(
4   i < until,
5   CodeBlock(
6     Declare(elem, ArrayRead(source, i)),
7     emit(elem),
8     Inc(i)
9   )
10 )

```

Le condizioni di terminazione accumulate durante l'enrichment vengono incorporate direttamente nella condizione del ciclo. Data C_s la condizione propria della sorgente e $p_1, \dots, p_n$ i predicati di terminazione, la condizione prodotta è:

$$C = C_s \wedge p_1 \wedge \dots \wedge p_n.$$

6.3.5 Specializzazione delle sorgenti

Anche la tipologia concreta della sorgente può influenzare il piano generato. Ad esempio, `java.util.List` beneficia di un controllo per distinguere il caso più favorevole di una `ArrayList`. Il piano assume quindi una struttura del tipo:

```

1 If(
2   IsInstanceOf[ArrayList](source),
3
4   // accesso indicizzato
5   ...,
6
7   // fallback tramite Iterator
8   ...
9 )

```

Nel caso di `ArrayList`, l'accesso indicizzato evita la creazione dell'iteratore. In caso di supporto ad `unsafe`, abilitato tramite la configurazione di compilazione, è inoltre possibile utilizzare l'accesso diretto all'array interno della collezione tramite `VarHandle`.

6.3.6 Generazione del piano parallelo

Nel caso di esecuzione parallela, la suddivisione del lavoro viene rappresentata esplicitamente nella OP IR attraverso l'operazione:

```
1 Parallel(  
2   returnSymbol,  
3   collectionSize,  
4   from,  
5   to,  
6   statements,  
7   localResult,  
8   combiner  
9 )
```

Il nodo rappresenta le informazioni necessarie a definire un lavoro partizionabile: per ciascun intervallo `[from,to)` deve essere eseguito lo stesso insieme di operazioni (`statements`), producendo eventualmente un risultato locale (`localResult`). La modalità con cui tali risultati devono essere ricombinati è rappresentata separatamente tramite una strategia di combinazione (`combiner`), mentre `returnSymbol` rappresenta il risultato finale della computazione parallela.

6.3.7 Risultato

Il risultato della traduzione è un `Program[T]`, composto da una sequenza di operazioni e da un valore finale:

```
1 case class Program[T](  
2   statements: List[Op],  
3   result: Value[T]  
4 )
```

A questo punto i concetti propri del dominio degli stream sono scomparsi: il loro comportamento è stato trasformato in dichiarazioni, condizioni, cicli, assegnamenti e operazioni sui valori.

6.4 Lowering

La fase di lowering costituisce l'ultimo passaggio del compilatore interno di FusedStream. Dato un `Program[T]` produce un'espressione Scala (`Expr[T]`), che viene restituita al compilatore e reinserita nel normale processo di compilazione.

Durante il lowering non vengono più prese decisioni riguardanti la struttura della pipeline. Abbiamo solo una traduzione del linguaggio OPIR nei corrispondenti nodi dell'AST Scala.

La traduzione è implementata attraverso due funzioni: `lowerOp`, che traduce le operazioni con effetti o controllo del flusso, e `lowerValue`, che traduce le espressioni che producono un valore.

6.4.1 Lowering delle operazioni

Le operazioni della OP IR vengono tradotte mediante pattern matching sul tipo concreto dell'operazione.

Le dichiarazioni sono convertite in `ValDef`:

```
1 case Declare(symbol, initialValue) =>  
2   ValDef(  
3     symbol,
```

```

4   Some(lowerValue(initialValue).asTerm)
5   )

```

mentre un assegnamento viene trasformato in un nodo `Assign`:

```

1  case AssignVal(symbol, value) =>
2    Assign(
3      Ref(symbol),
4      lowerValue(value).asTerm
5    )

```

Le strutture di controllo vengono invece ricostruite utilizzando le quote Scala. Ad esempio un nodo:

```

1  If(condition, thenBody, elseBody)

```

viene tradotto in una struttura della forma:

```

1  '{
2    if ($condition) {
3      $thenBody
4    } else {
5      $elseBody
6    }
7  }

```

Un `CodeBlock` viene infine trasformato in un nodo `quotes.reflect.Block`, traducendo ricorsivamente tutte le operazioni contenute.

6.4.2 Lowering dei valori

La traduzione dei `Value` segue lo stesso principio. Ogni nodo della gerarchia viene convertito in un `Expr[T]` corrispondente. I casi più semplici sono i riferimenti, le costanti e le espressioni Scala già disponibili:

```

1  case SymbolRef(symbol) => Ref(symbol).asExprOf[T]
2
3  case ScalaExpr(expr) => expr
4
5  case ConstantVal(expr) => expr

```

Gli operatori composti vengono invece tradotti ricorsivamente. Ad esempio:

```

1  LessThan(left, right)

```

diventa:

```

1  '{
2    ${lowerValue(left)} < ${lowerValue(right)}
3  }

```

Lo stesso procedimento viene applicato alle operazioni logiche, ai confronti, agli accessi ad array e collezioni e alle operazioni sui collector. La OP IR viene quindi progressivamente eliminata fino a ottenere esclusivamente nodi appartenenti all'AST Scala.

6.4.3 Conservazione delle informazioni di tipo

Per poter costruire un'Expr[T], le API di metaprogrammazione richiedono che sia disponibile un'evidenza Type[T].

Per questo motivo, come descritto nella Sezione 5.3, ciascun Value[T] conserva oltre al parametro di tipo anche il corrispondente valueType. Durante il pattern matching sulla OP IR tale informazione viene recuperata e resa disponibile localmente, ad esempio, nel lowering di una scrittura su array:

```
1 case write: ArrayWrite[t] =>
2   given Type[t] = write.valueType
3   val array = lowerValue[Array[t]](write.array)
4   val index = lowerValue[Int](write.index)
5   val value = lowerValue[t](write.value)
6   '{ $array($index) = $value }
```

La rappresentazione mantiene quindi il collegamento tra il sistema di tipi della IR e quello delle Scala Quotes fino all'ultima fase di generazione.

6.4.4 Applicazione e inlining delle funzioni

Un caso particolarmente importante è costituito da ApplyFun, che rappresenta l'applicazione di una funzione proveniente, ad esempio, da una map o da un filter. Qui viene utilizzata la β -reduction (descritta nella Sezione 3.3.3):

```
1 case app: ApplyFun[in, out] =>
2   given Type[in] = app.inType
3   given Type[out] = app.valueType
4
5   val arg = lowerValue[in](app.argument)
6
7   Expr.betaReduce(
8     '{ ${app.function}($arg) }
9   )
```

Se la funzione è una lambda direttamente disponibile alla macro, il risultato della β -reduction inserisce il corpo della funzione direttamente nel punto di utilizzo.

Ad esempio:

```
1 ApplyFun(
2   x => x * 2,
3   elem
4 )
```

Viene ridotto a:

```
1 elem * 2
```

Questo evita di utilizzare lambda (con le conseguenze note all'interno della JVM).

6.4.5 Lowering del parallelismo

Il nodo Parallel costituisce il caso meno diretto della fase di lowering. Il piano prodotto nella fase precedente ha già determinato la strategia di esecuzione e la modalità di combinazione. Il lowering trasforma tale descrizione nel codice concorrente effettivamente eseguibile. Il tipo di ParallelCombine determina quale implementazione specializzata viene utilizzata:

```

1 ParallelCombine.Sum(...)
2 ParallelCombine.ArrayConcat(...)
3 ParallelCombine.ArrayDirect()
4 ParallelCombine.GenericCombiner(...)

```

Nel caso di una somma, ad esempio, viene creato un array contenente un risultato parziale per ciascun chunk:

$$from_i = \left\lfloor \frac{i \cdot N}{C} \right\rfloor$$

$$until_i = \left\lfloor \frac{(i + 1) \cdot N}{C} \right\rfloor$$

dove N rappresenta la dimensione della sorgente e C il numero di chunk.

Il corpo del nodo `Parallel` viene quindi tradotto utilizzando tali valori come intervallo della sorgente e il risultato locale viene memorizzato nella posizione corrispondente:

```

1 partials(chunkIdx) = localResult

```

Al termine dell'elaborazione i risultati parziali vengono ridotti sequenzialmente per ottenere il risultato finale.

6.4.6 Scheduling dei chunk

Il numero di chunk viene determinato a runtime a partire dalla dimensione della sorgente e dalla configurazione:

```

1 if (size == 0) -> 0
2 else if (workerCount == 1) -> 1
3 else min(size, workerCount * chunksPerWorker)

```

I chunk non vengono assegnati staticamente ai worker: ogni worker riceve un chunk iniziale. Una volta terminato il proprio lavoro, richiede dinamicamente il chunk successivo tramite un contatore atomico condiviso.

6.4.7 Risultato del lowering

Al termine della fase, tutte le primitive appartenenti alla OP IR sono state eliminate e il risultato è costituito esclusivamente da un AST Scala tipizzato.

L'espressione ottenuta viene restituita dalla macro sotto forma di:

```

1 Expr [OUT]

```

e reinserita nel normale processo di compilazione Scala.

6.5 Esempio di traduzione end-to-end

Questa sezione ripercorre passo passo il processo di analisi e generazione del codice effettuato dalla libreria a partire da una semplice pipeline.

L'intero processo segue il flusso dell'applicativo:

Scala AST $\rightarrow$ StreamIR $\rightarrow$ Enriched StreamIR $\rightarrow$ Execution Plan $\rightarrow$ OpIR $\rightarrow$ Scala AST $\rightarrow$ JVM byte

6.5.1 L'input alla libreria

La pipeline utilizzata come esempio è mostrata nel Listato 6.5.1:

```
1 FusedStream.from(numbers)
2   .map(f)
3   .filter(p)
4   .toArray
```

La macro riceve l'espressione sotto forma di AST Scala tipizzato; in particolare, il parser riceve l'albero mostrato nel Listato 6.5.1:

```
1 Inlined(
2   None,
3   Nil,
4   Apply(
5     Select(
6       Apply(
7         Apply(
8           TypeApply(
9             Select(
10              Apply(
11                TypeApply(
12                  Select(Ident("FusedStream"), "from"),
13                  List(Inferred())
14                ),
15                List(Ident("numbers"))
16              ),
17              "map"
18            ),
19            List(Inferred())
20          ),
21          List(
22            Block(
23              List(
24                DefDef(
25                  "$anonfun",
26                  List(TermParamClause(List(ValDef("i", Inferred(), None)))),
27                  Inferred(),
28                  Some(
29                    Apply(
30                      Select(Ident("i"), "*"),
31                      List(Literal(IntConstant(2)))
32                    )
33                  )
34                )
35              ),
36              Closure(Ident("$anonfun"), None)
37            )
38          ),
39          ),
40          List(
41            Apply(
42              TypeApply(Ident("given_NotMapToStream_T"), List(Inferred())),
43              List(Ident("value"))
44            )
45          )
46        ),
```

```

47     "filter"
48   ),
49   List(
50     Block(
51       List(
52         DefDef(
53           "$anonfun",
54           List(TermParamClause(List(ValDef("i", Inferred(), None)))),
55           Inferred(),
56           Some(
57             Apply(
58               Select(
59                 Apply(
60                   Select(Ident("i"), "%"),
61                   List(Literal(IntConstant(2)))
62                 ),
63                 "=="
64               ),
65               List(Literal(IntConstant(0)))
66             )
67           )
68         ),
69       ),
70       Closure(Ident("$anonfun"), None)
71     )
72   )
73 )
74 )

```

E l'espressione rappresentate il terminale

```

1 fuse.FusedStream.toArray[scala.Int]

```

6.5.2 Il risultato del parsing

La prima fase consiste nella traduzione dell'AST Scala nella rappresentazione intermedia ad alto livello descritta nella Sezione 5.2. La Stream IR corrispondente all'AST del Listato 6.5.1 è riportata nel Listato 6.5:

Listing 6.5: Rappresentazione ottenuta al termine del parsing

```

1 Filter(
2   upstream = Map(
3     upstream = JListSource(
4       source = list,
5       elementType = Int
6     ),
7     function = (i: Int) => i * 2,
8     inputType = Int,
9     outputType = Int
10  ),
11  predicate = (i: Int) => i % 2 == 0,
12  elementType = Int
13 )

```

6.5.3 L'output dell'enrichment

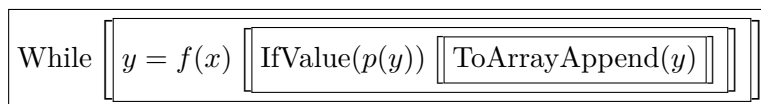
La StreamIR del Listato 6.5 viene quindi sottoposta alla fase di enrichment (Sezione 6.2). La Enriched StreamIR risultante è illustrata nel Listato 6.6:

Listing 6.6: StreamIR dopo la fase di enrichment

```
1 declarations = [  
2   // Dichiarazione della variabile dove viene materializzata la fonte  
3   Declare(source$macro$1, ScalaExpr(list)),  
4   // Dichiarazione della variabile che rappresenta la size della fonte  
5   Declare(sourceSize$macro$1, ScalaExpr(source$macro$1.size()))  
6 ]  
7  
8  
9 stream =  
10  Filter(  
11    upstream = Map(  
12      upstream = EnrichedJListSource(  
13        source = ScalaExpr(source$macro$1),  
14        size = ScalaExpr(sourceSize$macro$1),  
15        elementType = Int  
16      ),  
17      function = (i: Int) => i * 2,  
18      inputType = Int,  
19      outputType = Int  
20    ),  
21    predicate = (i: Int) => i % 2 == 0,  
22    elementType = Int  
23  )  
24  
25 streamInfo =  
26  StreamProperties(  
27    // Abbiamo un filter quindi la size non e' esatta ma un upper bound  
28    size = UpperBound(ScalaExpr(sourceSize$macro$1)),  
29    hasEarlyExit = false  
30  )
```

6.5.4 Generazione dell'OpIR

A partire dalla rappresentazione arricchita del Listato 6.6, il compilatore determina la strategia con cui attraversare la sorgente ed eseguire le operazioni della pipeline. Il piano di esecuzione assume la forma:



Il piano di esecuzione viene espresso tramite OpIR nel Listato 6.7:

Listing 6.7: OPIR generato per una pipeline con mapping, filtro e raccolta in array

```
1 statements = [  
2   // Estrae la lista di origine  
3   Declare(source$macro$1, ScalaExpr(numbers)),  
4   // Estrae la size della lista  
5   Declare(sourceSize$macro$1, ScalaExpr(source$macro$1.size())),  
6   // Crea un vettore preallocato
```

```

7  Declare(vec$macro$1, ArrayDefine[scala.Int](ScalaExpr(sourceSize$macro$1)),
8  // Dichiaro il counter e inizializzo a 0
9  Declare(i$macro$1, ConstantVal(0)),
10 // Genera codice ottimizzato a runtime se e' una attaylist usiamo accesso
    indicizzato (costo o(1), niente iteratori)
11 If(
12   condition = IsInstanceOf[java.util.ArrayList[]](ScalaExpr(source$macro$1)),
13   then =
14     CodeBlock(
15       Declare(i$macro$2, ConstantVal(0)),
16       While(
17         // Ciclo diretto su size, non abbiamo condizioni come limit o skip
18         condition = LessThan(SymbolRef(i$macro$2), ScalaExpr(sourceSize$macro$1)),
19         body =
20           CodeBlock(
21             // Leggiamo valore della list in una nuova variabile.
22             // Se useUnsafe = true allora JListRead legge direttamente
23             // l'array sottostante ad arraylist tramite varhandle
24             Declare(elem$macro$1,
25               JListRead(ScalaExpr(source$macro$1),
26                 SymbolRef(i$macro$2))),
27             // Map
28             CodeBlock(
29               Declare(
30                 mapped$macro$1,
31                 ApplyFun(((i: scala.Int) => i.*(2)), SymbolRef(elem$macro$1))
32               ),
33               // Filter
34               If(
35                 condition = ApplyFun(
36                   ((i: scala.Int) => i.%(2).==(0)),
37                   SymbolRef(mapped$macro$1)),
38                 then =
39                   // Se la condizione e' rispettata write all'indice
40                   CodeBlock(
41                     ArrayWrite(
42                       SymbolRef(vec$macro$1),
43                       SymbolRef(i$macro$1),
44                       SymbolRef(mapped$macro$1)),
45                     Inc(i$macro$1)
46                   )
47                 )
48               ),
49               Inc(i$macro$2)
50             )
51           )
52         ),
53     // codice non ottimizzato, per List senza accesso diretto, usa iteratore
54     else = ...
55   )
56 ],
57 result = IfValue(
58   // In base alla size dell'array lo ritorno direttamente o lo tronco
59   Equal(SymbolRef(i$macro$1), ArrayLength(SymbolRef(vec$macro$1))),
60   SymbolRef(vec$macro$1),
61   ArrayTake(SymbolRef(vec$macro$1), SymbolRef(i$macro$1))
62 )

```

6.5.5 Lowering verso l'AST Scala

L'elenco di operazioni viene poi tradotto nell'AST scala (qui omesso vista la verbosità e complessità). Segue invece una decompilazione del bytecode java corrispondente al codice generato:

```
1  ArrayList<Integer> numbers = new ArrayList<Integer>();
2  numbers.addAll(List.of(..));
3
4  int[] nArray;
5  RuntimeConfig conf = RuntimeConfig$.MODULE$.default();
6  ArrayList<Integer> source$macro$1 = numbers;
7  int sourceSize$macro$1 = source$macro$1.size();
8  int[] vec$macro$1 = new int[sourceSize$macro$1];
9  int i$macro$1 = 0;
10 if (source$macro$1 instanceof ArrayList) {
11     ArrayList<Integer> list$proxy1 = source$macro$1;
12     Object[] raw$macro$1 = ArrayListAccessor$.MODULE$.inline$ElementDataHandle().
    get(list$proxy1);
13     for (int i$macro$2 = 0; i$macro$2 < sourceSize$macro$1; ++i$macro$2) {
14         int elem$macro$1 = BoxesRunTime.unboxToInt(raw$macro$1[i$macro$2]);
15         int mapped$macro$1 = elem$macro$1 * 2;
16         if (mapped$macro$1 % 2 != 0) continue;
17         vec$macro$1[i$macro$1] = mapped$macro$1;
18         ++i$macro$1;
19     }
20 } else {
21     // tramite foreach, rimosso per brevit 
22 }
23 if (i$macro$1 == vec$macro$1.length) {
24     nArray = vec$macro$1;
25 } else {
26     Object object = Predef$.MODULE$.genericArrayOps(vec$macro$1);
27     nArray = (int[])ArrayOps$.MODULE$.take$extension(object, i$macro$1);
28 }
29 int[] result = nArray;
```

6.5.6 Il bytecode JVM generato

La corretta fusione delle operazioni pu  essere verificata anche ispezionando direttamente il bytecode JVM generato (riportato nel Listato 6.8):

Listing 6.8: Estratto del bytecode JVM relativo al corpo della pipeline

```
1  125: iload      10
2  127: iload      5
3  129: if_icmpge    175
4
5  132: aload      8
6  134: iload      10
7  136: aaload
8  137: invokestatic BoxesRunTime.unboxToInt
9  140: istore     11
10
11 142: iload      11
12 144: iconst_2
13 145: imul
14 146: istore     12
```

```

15
16 148: iload      12
17 150: iconst_2
18 151: irem
19 152: iconst_0
20 153: if_icmpne  169
21
22 156: aload      6
23 158: iload      7
24 160: iload      12
25 162: iastore
26 163: iinc       7, 1
27
28 169: iinc       10, 1
29 172: goto       125

```

Il risultato mostra in maniera evidente l'effetto della fusione e dell'inlining. L'operazione di mapping

$$x \mapsto x \cdot 2$$

non corrisponde più all'invocazione di una funzione, ma viene tradotta direttamente nell'istruzione JVM `imul`. Analogamente, il predicato

$$x \bmod 2 = 0$$

è rappresentato dalle istruzioni `irem` e `if_icmpne`.

Nel corpo del ciclo non sono presenti invocazioni dinamiche verso le funzioni utilizzate da `map` e `filter`. Non compaiono né chiamate a `Function.apply`, né istruzioni `invokedynamic` associate alla costruzione o all'invocazione di lambda. Anche la raccolta del risultato è ridotta a operazioni primitive: `iastore` scrive direttamente il valore trasformato nell'array di output e `iinc` aggiorna il relativo indice.

Capitolo 7

Validazione e valutazione

7.1 Strategia di testing

Il testing di una libreria come `FusedStream` deve affrontare due problemi:

1. **Correttezza a runtime:** accertarsi che il codice generato dalle macro produca i risultati attesi, rispetti l'ordine degli elementi e preservi la semantica delle normali collezioni Scala.
2. **Verifica a compile-time:** assicurarsi che le combinazioni non valide vengano intercettate subito dal compilatore, fornendo messaggi di errore chiari ed evitando comportamenti anomali o errori oscuri a runtime.

I test sono scritti con `MUnit` e sono organizzati su più livelli separando i controlli interni da quelli sull'API pubblica:

- **Unit test interni:** verificano i singoli stadi del motore di compilazione in isolamento.
- **Test funzionali end-to-end:** compilano ed eseguono pipeline complete scritte tramite l'API pubblica, verificando il risultato dell'esecuzione. Questi test sono suddivisi in quattro suite dedicate:
 - `SequentialStreamTest`: trasformazioni sequenziali di base e operatori di slice;
 - `PipelineSemanticsTest`: ordine di valutazione delle espressioni, isolamento dei side-effect e correttezza della fusione;
 - `SourceAndCollectorTest`: sorgenti Scala e Java, ottimizzazioni con accesso diretto alla memoria e collector terminali;
 - `ParallelStreamTest`: partizionamento del carico su più worker, concorrenza e riduzioni parallele.
- **Test di compilazione:** verificano che l'uso scorretto dell'API venga bloccato a tempo di compilazione.

Metriche di copertura

La copertura del codice è stata misurata tramite `sbt-scoverage` sul modulo (`core`), come riassunto in Tabella 7.1. La suite raggiunge una copertura complessiva delle istruzioni del **95.05%** (1708 su 1797 istruzioni) e una copertura dei rami decisionali del **94.67%** (320 su 338 rami).

Tabella 7.1: Copertura del codice rilevata da `sbt-scoverage` sul modulo `core`.

Package / Sottosistema	Istruzioni	Coverage	Branch coverage
<code>fuse.internal.parallel</code>	100.00%	100.00%	
<code>fuse.internal.ir</code>	100.00%	100.00%	
<code>fuse.internal</code>	95.19%	92.89%	
<code>fuse (API pubblica)</code>	68.75%	83.33%	
Totale	1708 / 1797	95.05%	94.67%

7.2 Analisi del codice generato

7.2.1 Fusione delle operazioni

7.2.2 Inlining delle lambda

7.2.3 Boxing e allocazioni

7.2.4 Analisi del bytecode

7.3 Valutazione delle prestazioni

7.3.1 Metodologia

7.3.2 Baseline

7.3.3 Benchmark sequenziali

7.3.4 Benchmark paralleli

7.3.5 Allocazioni

Capitolo 8

Retrospettiva

Bibliografia

- [1] Duncan Coutts, Roman Leshchinskiy, and Don Stewart. Stream fusion: From lists to streams to nothing at all. *ACM SIGPLAN Notices*, 42(9):315–326, 2007.
- [2] Oleg Kiselyov, Aggelos Biboudis, Nick Palladinos, and Yannis Smaragdakis. Stream fusion, to completeness. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, pages 285–299, 2017.
- [3] Shayan Najd and Simon Peyton Jones. Trees that grow. *Journal of Universal Computer Science*, 23(1), 2017.
- [4] Nicolas Stucki, Jonathan Immanuel Brachthäuser, and Martin Odersky. Multi-stage programming with generative and analytical macros. In *Proceedings of the 20th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences*, pages 110–122, 2021.
- [5] Nicolas Alexander Stucki. *Scalable metaprogramming in scala 3*. PhD thesis, EPFL, 2023.